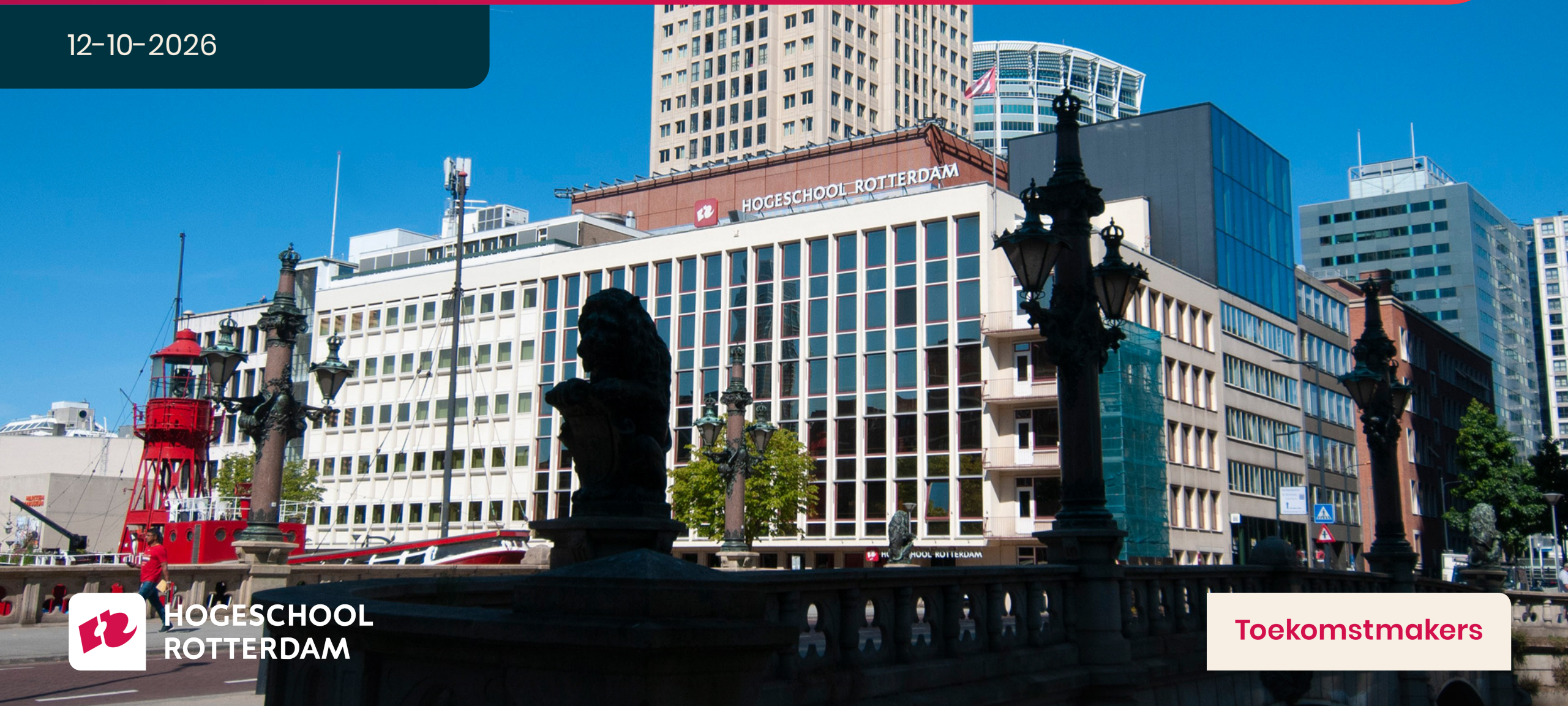
Technische Informatica: High-performance Computing

Les 16: Introductie shaders

12-10-2026

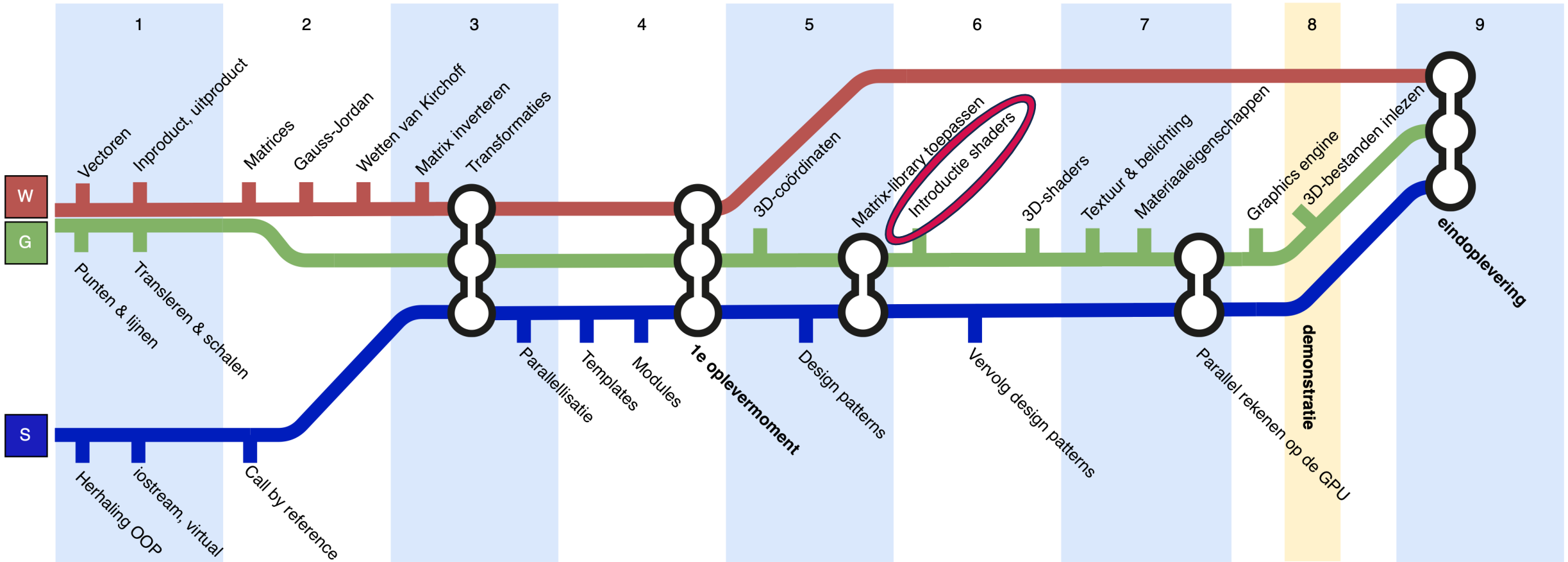


HOGESCHOOL ROTTERDAM

Toekomstmakers

Quiz

Metrokaart


1
2
3
4
5
6
7
8
9
W
G
S
Vectoren
Inproduct, uitproduct
Matrices
Gauss-Jordan
Wetten van Kirchoff
Matrix inverteren
Transformaties
3D-coördinaten
Matrix-library toepassen
Introductie shaders
3D-shaders
Textuur & belichting
Materiaaleigenschappen
Graphics engine
3D-bestanden inlezen
eindoplevering
Punten & lijnen
Transleren & schalen
Herhaling OOP
iostream, virtual
Call by reference
Parallellisatie
Templates
Modules
1e oplevermoment
Design patterns
Vervolg design patterns
Parallel rekenen op de GPU
demonstratie

Leerdoelen

De student kan:

1. bewerkingen met vectoren en matrices uitvoeren en deze toepassen in verschillende contexten zoals computer graphics en neurale netwerken.
2. object-georiënteerde programmeertechnieken toepassen in de programmeertaal C++.
3. shader-programma's schrijven en uitvoeren op een GPU in GLSL.
4. relevante design patterns onderbouwd selecteren en toepassen.



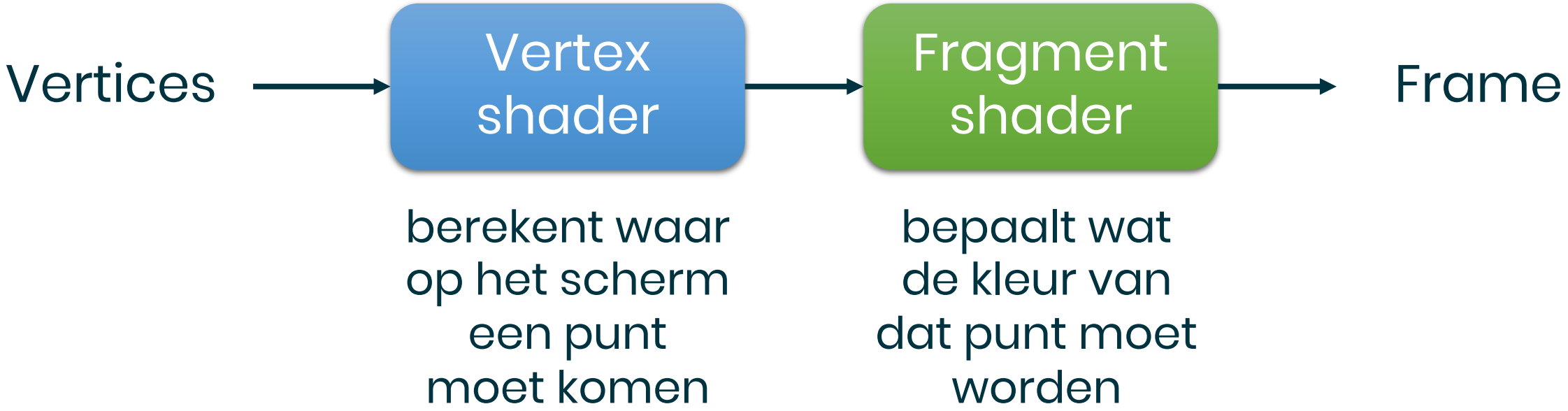
Planning

1. Fragment shaders
2. Vertex shaders



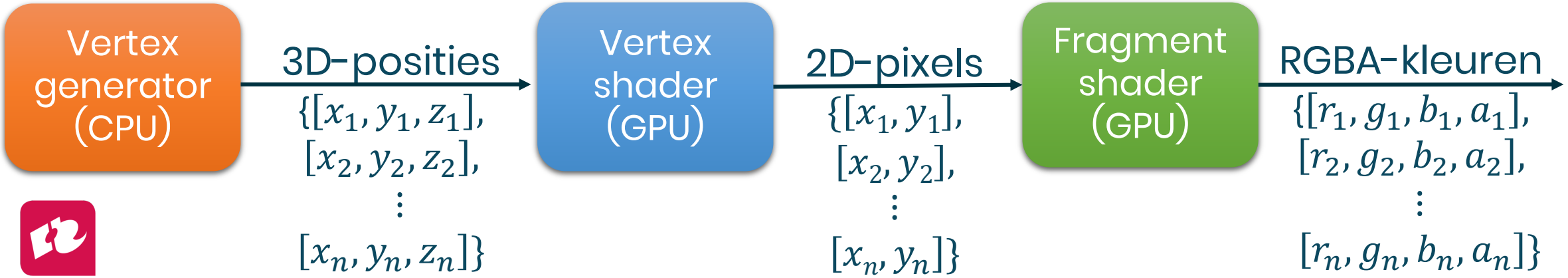
Concept: Graphics *pijplijn*

- **Input**: lijst 3D-hoekpunten (vertices)
- **Output**: gerenderd plaatje (frame)



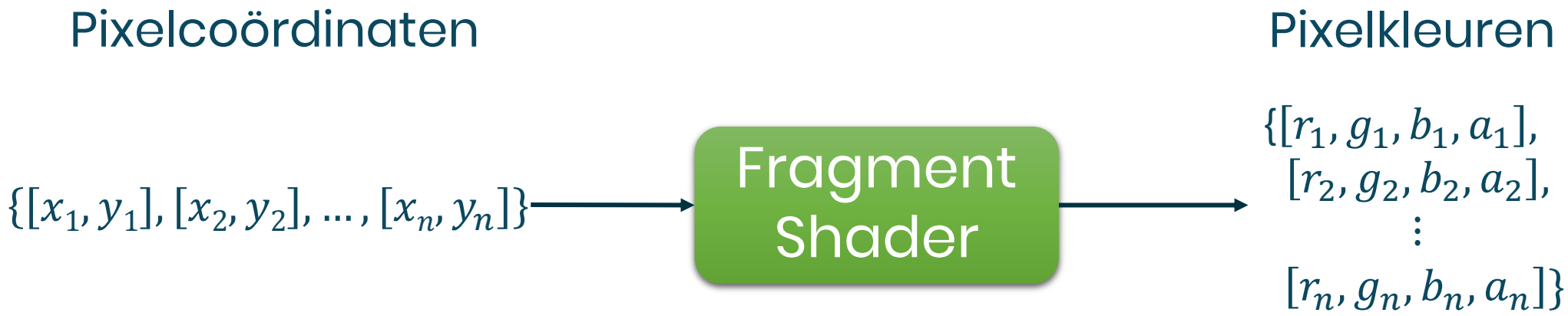
Shaders

- Programma's die draaien op de GPU in plaats van de CPU.
- Geschreven in shader-taal GLSL (lijkt op C).
- **Vertex shader**:
  - Krijgt een 3D-punt in de ruimte en geeft 2D-pixelcoördinaten terug.
  - Wordt toegepast op alle vertices van wat we willen tekenen.
- **Fragment shader**:
  - Krijgt 2D-pixelcoördinaten en geeft een RGBA-kleur terug.
  - Wordt toegepast op alle pixels van het *fragment* (punt, lijn of driehoek).



Fragment shader

- Functie die de *shade* van ieder pixel bepaalt.
- Ook wel *pixel shader* genoemd.
- Moet schrijven naar variabele `fragColor` (een 4-vector: r, g, b, alpha).
- Kan parallel draaien op meerdere cores van de GPU die elk een eigen gebied van het scherm bedienen.



Oefening: Fragment shader in ShaderToy

- Ga naar https://www.shadertoy.com/, een platform om (fragment) shaders te testen en ermee te spelen.
- Klik op “new” om een nieuwe shader te maken.
- Plak deze code (zie GitLab) en klik op het driehoekje om te runnen.

```
vec4 circle(vec2 p) {
  const float r = 100.0; // radius
  if (p.x * p.x + p.y * p.y <= r * r) {
    return vec4(1, 0, 0, 1); // red
  }
  return vec4(0, 1, 0, 1); // green
}
void mainImage(out vec4 fragColor, in vec2 fragCoord) {
  vec2 c = fragCoord - 0.5 * iResolution.xy; // shift origin to centre
  fragColor = circle(c);
}
```



Oefening: Fragment shader in ShaderToy

- Probeer in plaats van `circle()` ook `checkBoard()`:

```
vec4 checkBoard(vec2 c) {
  const float s = 50.0; // size
  vec2 p = floor(c / s);
  if (int(p.x + p.y) % 2 == 0) {
    return vec4(1, 0, 0, 1); // red
  }
  return vec4(0, 1, 0, 1); // green
}
```

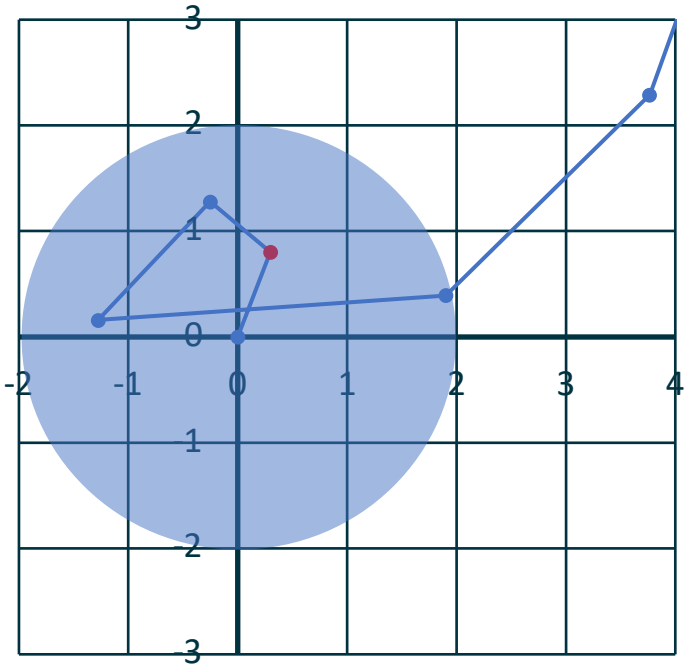
- Kun je in plaats van een cirkel ook andere vormen maken, bijv. een vierkant of een driehoek?



Demo: Mandelbrot-set

- **Mandelbrot-set**: een soort fractal (zelf-herhalend patroon).
- Een pixel valt ofwel binnen de set, ofwel er buiten.
- Voor ieder pixel apart te berekenen:

1. Begin met het pixel $\vec{c} = (c_x, c_y)$ en een punt $\vec{z} = (0, 0)$.
2. Bereken een nieuwe $\vec{z} = (z_x{}^2 - z_y{}^2 + c_x, 2 \cdot z_x \cdot z_y + c_y)$.
3. Als $|z| > 2$, dan valt punt $\vec{c}$ buiten de set ⇒ stop.
4. Ga terug naar stap 2.

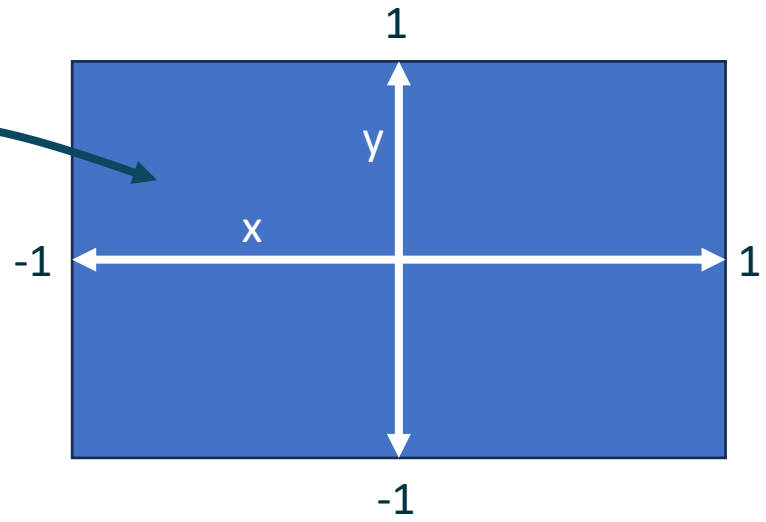


- Als je deze loop een aantal keer (~100) gedaan hebt, dan is het waarschijnlijk dat punt c binnen de set valt ⇒ stop.

Pauze (15min)

Vertex shader

- Geeft aan welke **positie** een *fragment* moet krijgen.
- Moet schrijven naar speciale variabele `gl_Position`.
- Vector van 4 componenten:
  `gl_Position = vec4(x, y, z, w);`
- `x` en `y`: positie in *clipping space*.
- `z`: voor *depth buffer* (komt later).
- `w`: hier wordt alles door gedeeld;
  gebruik `1.0` voor nu.



3D-posities

$\{[x_1, y_1, z_1],$
$[x_2, y_2, z_2],$
$\vdots$
$[x_n, y_n, z_n]\}$

Vertex shader

2D-posities

$\{[x'_1, y'_1, 0, 1],$
$[x'_2, y'_2, 0, 1],$
$\vdots$
$[x'_n, y'_n, 0, 1]\}$

Shader workflow

Een C++ programma dat gebruik maakt van shaders moet:

1. Zorgen voor een OpenGL *context* (het venster waarin gerenderd wordt).
2. Een GLSL shader-programma compileren om op de grafische kaart uitgevoerd te worden door de GPU.
3. De data (vertices) ook op de grafische kaart zetten.
4. De data koppelen aan bepaalde variabelen ("*attributes*") in het shader-programma.
5. Een tekeninstructie geven om het shader-programma te laten uitvoeren.



GLFW-library

- Zorgt voor de *context* waarin je GL-functies kunt gebruiken.
- Maakt een venster en handelt de input van muis en toetsenbord af.
- Hebben we al gebruikt in de draw-library.

```
#include <GLFW/glfw3.h>
 ⋮
glfwInit();
```

- Voor shaders hebben we een *core profile* context nodig met versie 3.3:

```
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
GLFWwindow* window = glfwCreateWindow(640, 480, "Shader example", NULL, NULL);
glfwMakeContextCurrent(window);
```



GLEW-library

- Omdat ieder GPU-model weer andere mogelijkheden (extensions) heeft, moet er een tussenlaag zijn die ervoor zorgt dat de juiste functies beschikbaar zijn.
- Dit is wat de GLEW-library (GL Extension Wrangler) doet.
- Installatie op Windows: `pacman -S mingw-w64-ucrt-x86_64-glew`
- Gebruik:

```
#include <GL/glew.h>
#include <GLFW/glfw3.h>
⋮
// after context is available:
glewInit()
```



Vertex shader

- Shader code moet beginnen met *version string*: `#version 330 core`
- Kan input krijgen: *attribute*
  - Declareer *attribute* met keyword `in`.
  - `pos` wordt later ingevuld.
- Moet schrijven naar speciale variabele **`gl_Position`**.
- Doet alle berekeningen voor transformaties en projectie (dit voorbeeld nog niet).

```
#version 330 core

in vec3 pos;

void main() {
  gl_Position = vec4(pos, 1.0);
}
```



Vertex shader compileren

- De source code van de shader is opgenomen in een *raw string* of kan als apart bestand ingelezen worden.
- *Raw string*:
  - begint met `R"(` en eindigt met `)"`
  - geen escape-codes
  - newlines worden letterlijk overgenomen

```
GLuint vs = glCreateShader(GL_VERTEX_SHADER);
const char *vsSource =
R"(#version 330 core
   in vec3 pos;
   void main() {
     gl_Position = vec4(pos, 1.0);
   })";
glShaderSource(vs, 1, &vsSource, NULL);
glCompileShader(vs);
```



Fragment shader

- Moet een output variabele van type `vec4` definiëren met keyword `out`.

```
GLuint fs = glCreateShader(GL_FRAGMENT_SHADER);
const char *fsSource =
R"(#version 330 core
   out vec4 fragColor;
   void main() {
     fragColor = vec4(0.5, 0.5, 1.0, 1.0);
   })";
glShaderSource(fs, 1, &fsSource, NULL);
glCompileShader(fs);
```

- Deze shader definieert een kleur (0.5, 0.5, 1.0; lichtblauw) en een opaciteit (1.0; ondoorzichtig) en kent die toe aan `fragColor`.
- Dit geeft dus dezelfde kleur voor alle fragments (pixels) van de figuur.



Shader program

- De twee shaders moeten gecombineerd worden tot één *shader program*:

```
GLuint program = glCreateProgram();
glAttachShader(program, vs);
glAttachShader(program, fs);
glLinkProgram(program);
glUseProgram(program);
```



Data

- De data voor de vertices moet naar een *Vertex Buffer Object* (VBO) in het geheugen van de GPU gekopieerd worden.
- Hiervoor hebben we ook een *Vertex Array Object* (VAO) nodig, waarin bijgehouden wordt hoe de data gestructureerd is.
- Je kan meerdere VAO's en VBO's hebben.
- Om aan te geven welke je wilt gebruiken, moet je ze *binden*.

```
GLfloat vertices[] = {
  -1,  0, 0,
   0,  1, 0,
   1, -1, 0
};
GLuint vao;
glGenVertexArrays(1, &vao);
glBindVertexArray(vao);
GLuint vbo;
glGenBuffers(1, &vbo);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER,
  sizeof(vertices), vertices,
  GL_STATIC_DRAW);
```



Data koppelen

- De data in de VBO moet aan een variabele (“*attribute*”) in het shaderprogramma gekoppeld worden.
- Nodig: de *locatie* van het attribute in het geheugen:

```
GLint posLocation = glGetAttribLocation(program, "pos");
glVertexAttribPointer(posLocation, 3, GL_FLOAT, GL_FALSE, 0, 0);
glEnableVertexAttribArray(posLocation);
```

- 3 getallen per vertex voor dit attribute (x, y en z)
- Getaltype is `float`
- Niet normaliseren (n.v.t. bij `float`)
- Standaard *stride* (totaal aantal bytes per vertex)
- Geen offset



Tekenen

- In een loop wordt het hele plaatje telkens opnieuw getekend.
- Daarna wachten totdat een *event* optreedt (bijv. window geresized, muisklik, toetsaanslag) .

```
while (!glfwWindowShouldClose(window)) {
  glClear(GL_COLOR_BUFFER_BIT);
  glDrawArrays(GL_LINE_LOOP, 0, 3);
  glfwSwapBuffers(window);
  glfwWaitEvents();
}
```

- GL_LINE_LOOP geeft aan hoe de punten getekend worden.
- 3 is het aantal te tekenen vertices.



Opdracht: Mijn eerste shader

- Installeer de GLEW library:

Windows / MSYS2

```
pacman -S mingw-w64-ucrt-x86_64-glew
```

MacOS

```
brew install glew
```

- Compileer het voorbeeldprogramma (zie GitLab).
- Probeer als tekeninstructie ook `GL_TRIANGLES` in plaats van `GL_LINE_LOOP`.
- Pas het programma aan zodat er meerdere driehoeken getekend worden.
- Kun je ook andere vormen dan driehoeken tekenen, bijv. vierhoeken? Tip: `GL_TRIANGLE_STRIP`. Zie de OpenGL-documentatie.



Check-out

- Wat hebben we geleerd?
  - Wat een fragment shader is.
  - Wat een vertex shader is.
  - Hoe je shaders compileert en uitvoert vanuit C++.



Afsluiting

Verwerkingsopdracht:

- Maak een **Shader**-class met een constructor waaraan je de shader-code kan meegeven, en een method voor het compileren van de shader.
- Maak subclasses voor een vertex shader en een fragment shader.
- Pas het voorbeeldprogramma aan zodat het deze classes gebruikt.

Voorbereidingsopdracht:

- Zoek uit wat de verschillen en overeenkomsten zijn tussen:
  - Een Vertex Array Object (VAO)
  - Een Vertex Buffer Object (VBO)
  - Een Element Buffer Object (EBO)

Volgende keer:

- Vervolg 3D-shaders



HOGESCHOOL ROTTERDAM

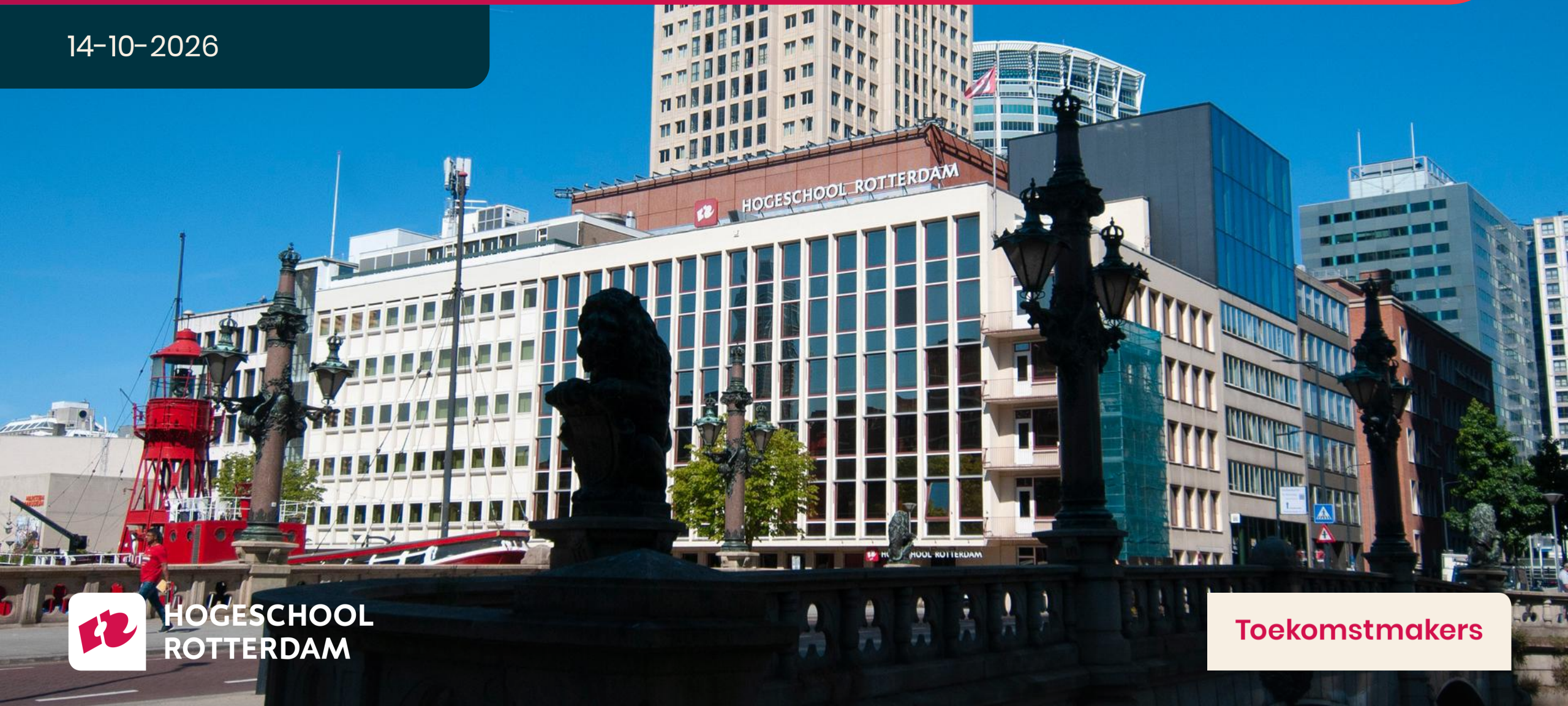
Toekomstmakers

© 2025 Hogeschool Rotterdam, alle rechten voorbehouden

Technische Informatica: High-performance Computing

Les 17: 3D shaders

14-10-2026



HOGESCHOOL ROTTERDAM

Toekomstmakers

Metrokaart


1
2
3
4
5
6
7
8
9
W
G
S
Vectoren
Inproduct, uitproduct
Matrices
Gauss-Jordan
Wetten van Kirchoff
Matrix inverteren
Transformaties
3D-coördinaten
Matrix-library toepassen
Introductie shaders
3D-shaders
Textuur & belichting
Materiaaleigenschappen
Graphics engine
3D-bestanden inlezen
eindoplevering
Punten & lijnen
Transleren & schalen
Parallellisatie
Templates
Modules
1e oplevermoment
Design patterns
Vervolg design patterns
Parallel rekenen op de GPU
demonstratie
Herhaling OOP
Iostream, virtual
Call by reference

Leerdoelen

De student kan:

1. bewerkingen met vectoren en matrices uitvoeren en deze toepassen in verschillende contexten zoals computer graphics en neurale netwerken.
2. object-georiënteerde programmeertechnieken toepassen in de programmeertaal C++.
3. shader-programma's schrijven en uitvoeren op een GPU in GLSL.
4. relevante design patterns onderbouwd selecteren en toepassen.

Planning

1. 3D-shaders (opdrachten)



Bespreken voorbereidingsopdracht

- Maak met de klas een gezamenlijke lijst van verschillen en overeenkomsten van een VAO, een VBO en een EBO.

Opdrachten: 3D shaders

- **Zie instructies op GitLab**.

1. **Draadmodel kubus**: combineer de kubus van les 13 met de shaders van les 16

2. **Kubus met dichte oppervlakken**: ga van draadmodel naar dichte oppervlakken

3. **Kubus met diepte-informatie**: zorg dat de kubus juist getekend wordt door diepte-informatie mee te geven



Check-out

- Wat hebben we geleerd?
  - Hoe je een 3D-figuur tekent met een shader-programma.
  - Hoe je kleur toevoegt aan oppervlakken in een shader.
  - Hoe je diepte-informatie toevoegt aan een shader.



Afsluiting

Verwerkingsopdracht:

- Zorg dat je alle drie de opdrachten af hebt en getest.

Voorbereidingsopdracht voor les 18 (vrijdag 16 okt.):

- Maak een klassendiagram van je graphics engine tot nu toe in UML.

Voorbereidingsopdracht voor les 19 (maandag 26 okt.):

- Noteer voor jezelf over de opdrachten:
  - Wat is een attribute? Welke attributes heb je in je programma?
  - Wat is een uniform? Welke uniforms heb je in je programma?
  - Wat is een vertex array object (VAO)? Wat is de functie hiervan in je programma?
  - Wat is een vertex buffer object (VBO)? Wat voor soort data staat hierin?
  - Wat is een element buffer object (EBO)? Wat voor soort data staat hier in?
  - Wat is de depth buffer en hoe gebruik je die in je programma?

Volgende keer:

- Design patterns (vervolg)



HOGESCHOOL ROTTERDAM

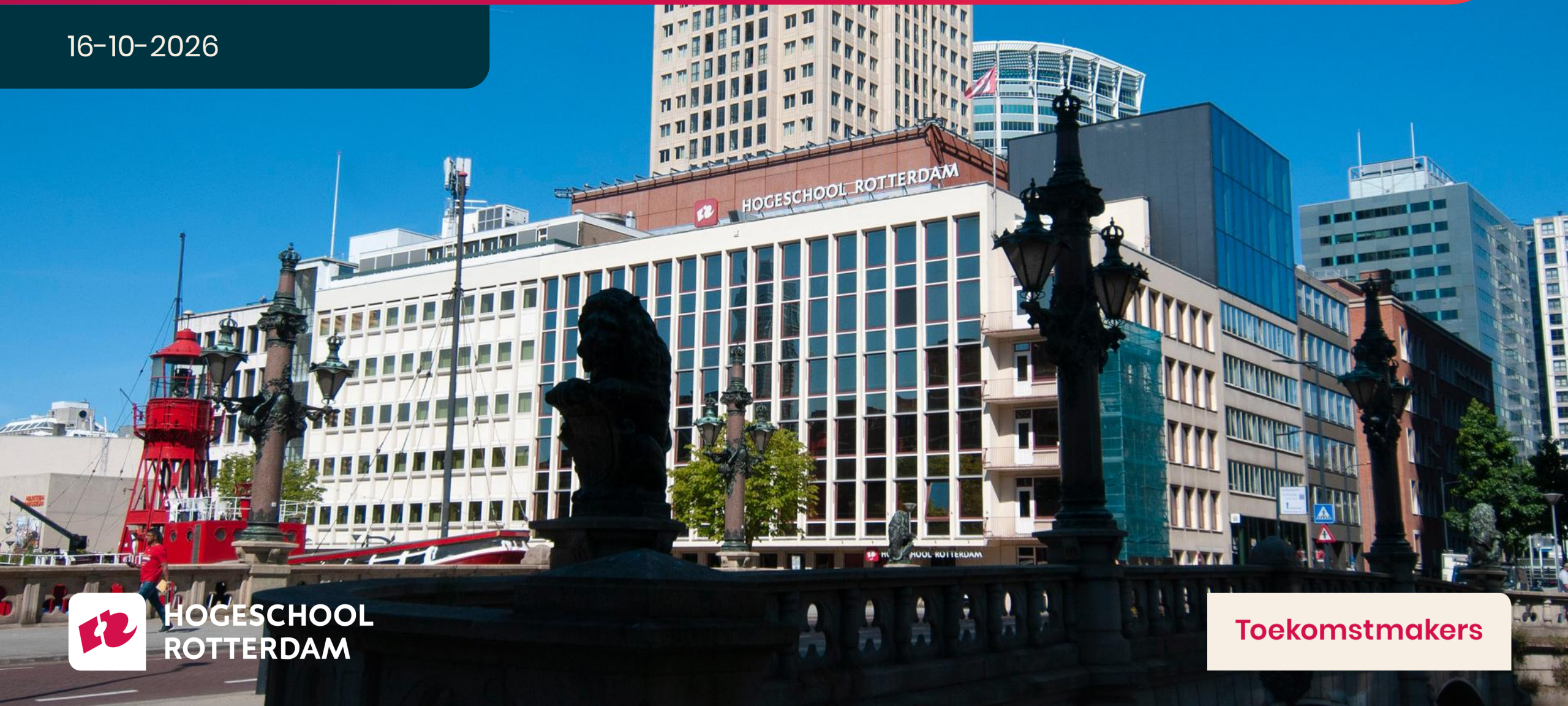
Toekomstmakers

© 2025 Hogeschool Rotterdam, alle rechten voorbehouden

Technische Informatica: High-performance Computing

Les 18: Design Patterns vervolg

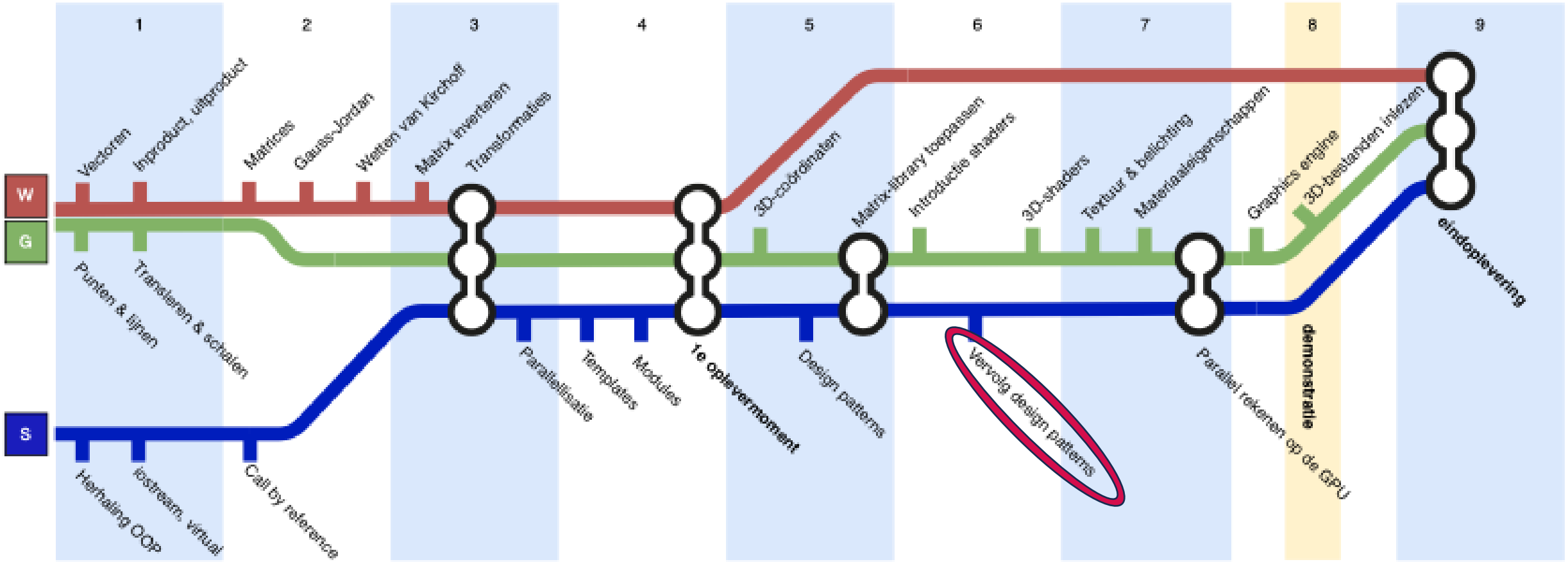
16-10-2026



HOGESCHOOL ROTTERDAM

Toekomstmakers

Metrokaart


1
2
3
4
5
6
7
8
9
W
G
S
Vectoren
Inproduct, uitproduct
Matrices
Gauss-Jordan
Wetten van Kirchoff
Matrix inverteren
Transformaties
3D-coördinaten
Matrix-library toepassen
Introductie shaders
3D-shaders
Textuur & belichting
Materiaaleigenschappen
Graphics engine
3D-bestanden inlezen
eindoplevering
Punten & lijnen
Transleren & schalen
Herhaling OOP
Iostream, virtual
Call by reference
Parallellisatie
Templates
Modules
1e oplevermoment
Design patterns
Vervolg design patterns
Parallel rekenen op de GPU
demonstratie

Leerdoelen

De student kan:

1. bewerkingen met vectoren en matrices uitvoeren en deze toepassen in verschillende contexten zoals computer graphics en neurale netwerken.
2. object-georiënteerde programmeertechnieken toepassen in de programmeertaal C++.
3. shader-programma's schrijven en uitvoeren op een GPU in GLSL.
4. relevante design patterns onderbouwd selecteren en toepassen.



Planning

1. Het singleton patroon
2. Het adapter patroon



Bespreken voorbereidingsopdracht

Is het gelukt om de strategy te implementeren?

Is het gelukt om een class diagram te maken van de graphics engine tot nu toe.



Het singleton patroon

naam: het singleton patroon

doel 1: garanderen dat een class maar één instantie heeft

doel 2: globale toegang naar die ene instantie

hoe: een private constructor en een static functie voor toegang tot de instantie

gevolgen: garantie van uniekheid en globale toegang



Het singleton patroon

Stel, we hebben een applicatie waarin verschillende onderdelen instellingen uit een bestand(bijv. settings.json) moeten inlezen.



Het singleton patroon

Als ieder onderdeel een nieuw object maakt van de klasse ConfigSettings krijg je twee problemen:

Het onnodig lezen het bestand

Als een gebruiker een instelling wijzigt in een klasse, dan weten de andere klasse dat niet omdat ze hun eigen kopie van de instellingen in het geheugen hebben.



Het singleton patroon

Door gebruik te maken van het Singleton patroon, kan er nog maar 1 instantie van de instellingen bestaan in het programma.



Het singleton patroon

If bestaat niet -> maak nieuwe en geef terug

Else bestaat -> geef terug



Het singleton patroon

Waar zou dit van pas kunnen komen in de graphics engine?

Onderdelen van het programma die resources willen gebruiken, bijvoorbeeld de GPU-buffer of het scherm

Een asset-manager die bijhoudt welke shaders/textures geladen zijn om dubbele laadactie te voorkomen.



Het singleton patroon

Opletten:

Bij multithreading moet de object creatie tread-safe gemaakt worden.

Wat nou als we meerdere schermen willen gebruiken? Dan hebben we toch meerdere instanties nodig en zullen we de code moeten herschrijven.

Als dat zo is, dan moet je overwegen of je het singleton patroon wel moet gebruiken.



Het adapter patroon

naam: het adapter patroon

doel: de client applicatie laten samenwerken met een ingewikkeld systeem zonder dit systeem te hoeven aanpassen

hoe: bouw een adapter tussen systeem en client applicatie, de adapter is een soort vertaler

gevolgen: bestaande systemen kunnen worden uitgebreid zonder rekening te hoeven houden met de interface



Het adapter patroon – Voorbeeld 1

In een groot ingewikkeld systeem wordt gewerkt met temparatuur. . .

Het systeem is in de VS gebouwd en wordt in de VS gebruikt.

Het systeem voert berekeningen uit in graden Fahrenheit.

Onze applicatie moet gekoppeld worden aan het bestaande Amerikaanse systeem.

De opdrachtgever van onze applicatie is Europees en werkt met graden Celsius.



Het adapter patroon - Voorbeeld 1

Bouw een adapter tussen het bestaande systeem en de te bouwen applicatie.

De adapter vertaalt elke temparatuur in Celsius naar Fahrenheit en vice versa.

Te koppelen applicatie <-> Adapter <-> Bestaande systeem



Het adapter patroon – Voorbeeld 2

Blender, een 3d modelleringsprogramma. . .

Werking:

- bouw een 3d model
- smeer bitmaps tegen alle vlakken
- render het model



Het adapter patroon – Voorbeeld 2

Een groep slimmeriken is bezig met het bouwen van een nieuwe rendering engine. . .

de engine bestaat uit een enorme hoeveelheid classes met daarin vele methodes

men wil blender met deze engine laten werken

de technici werken vrolijk verder aan hun engine en vragen de heren van Blender niets

Probleem: laat Blender met deze nieuwe engine werken



Het adapter patroon – Voorbeeld 2

Bouw een adapter:

de adapter klasse heeft een pointer naar het rendering object in zich.

methodes die vanuit Blender worden aangeroepen, worden door de adapter vertaald naar methodes die de engine begrijp



Het adapter patroon – Voorbeeld 3

Diverse programma’ s moeten kunnen reageren op input van het toetsenbord of de muis.



Het adapter patroon – Voorbeeld 3

Diverse programma's moeten kunnen reageren op input van het toetsenbord of de muis.

De ene muis is echter de andere niet. . .

Het ene toetsenbord is ook het andere niet. . .

In Java heeft men een oplossing reeds in de Java A.P.I. zitten:

https://docs.oracle.com/en/java/javase/27/docs/api/java.desktop/java/awt/event/MouseAdapter.html

https://docs.oracle.com/en/java/javase/27/docs/api/java.desktop/java/awt/event/KeyAdapter.html



Opdracht

Schrijf een programma met een thermostaat die een waardes in Fahrenheit terug geeft. (interface: `float fahrenheit();`)

De sensor die we hebben geeft een waarde in Celsius terug.

Schrijf een Adapter die de interface van de thermostaat implementeert en `fahrenheit()` kan aanroepen, zonder dat de gebruiker weet dat er een Celsius sensor achter zit.



Check-out

- Wat hebben we geleerd?
  - Het Singleton patroon
  - Het Adapter patroon



Afsluiting

Verwerkingsopdracht:

- Maak de opdracht af.

Voorbereidingsopdracht:

- Zie slides van les 17 (woensdag 14 okt.).

Volgende keer:

- Textuur
- Belichting

HOGESCHOOL ROTTERDAM

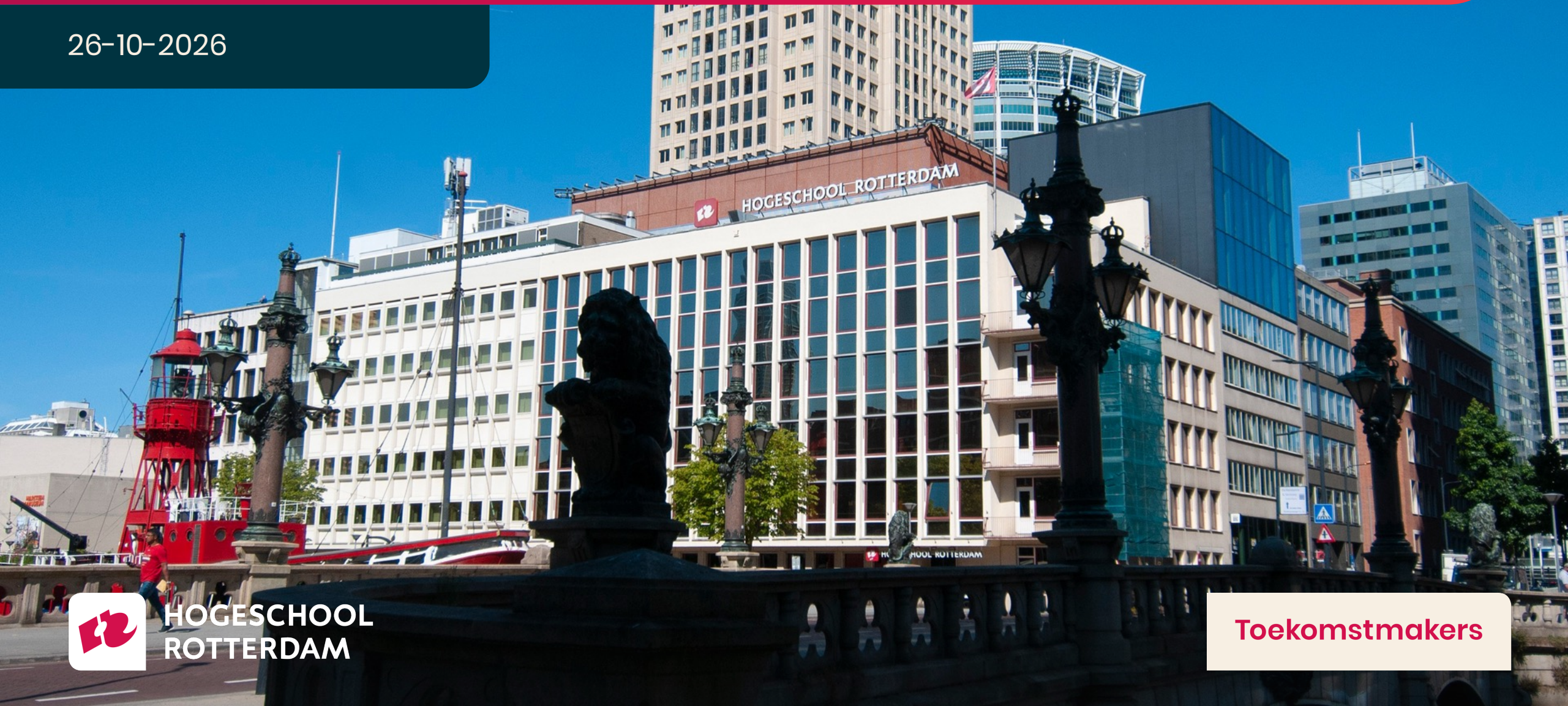
Toekomstmakers

© 2025 Hogeschool Rotterdam, alle rechten voorbehouden

Technische Informatica: High-performance Computing

Les 19: Textuur & belichting

26-10-2026

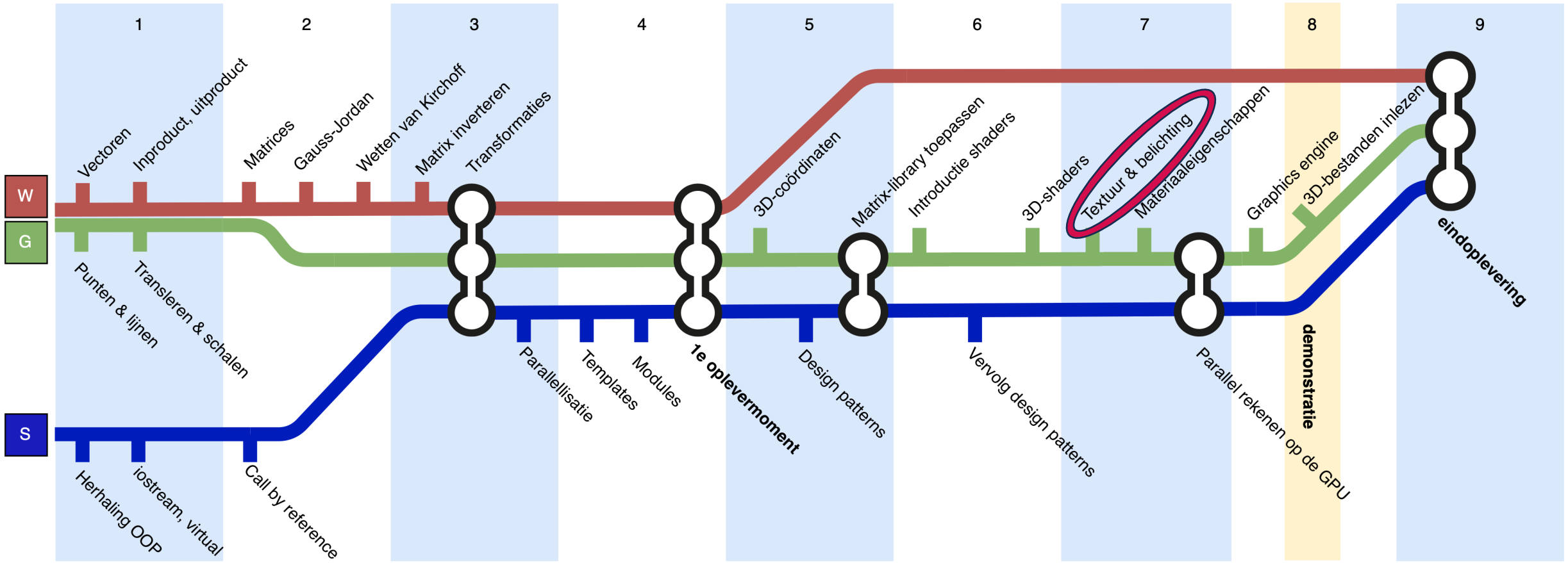


HOGESCHOOL ROTTERDAM

Toekomstmakers

Quiz

Metrokaart


1
2
3
4
5
6
7
8
9
W
G
S
Vectoren
Inproduct, uitproduct
Matrices
Gauss-Jordan
Wetten van Kirchoff
Matrix inverteren
Transformaties
Punten & lijnen
Transleren & schalen
Herhaling OOP
iostream, virtual
Call by reference
Parallellisatie
Templates
Modules
1e oplevermoment
3D-coördinaten
Design patterns
Matrix-library toepassen
Introductie shaders
Vervolg design patterns
3D-shaders
Textuur & belichting
Materiaaleigenschappen
Parallel rekenen op de GPU
Graphics engine
3D-bestanden inlezen
demonstratie
eindoplevering

Leerdoelen

De student kan:

1. bewerkingen met vectoren en matrices uitvoeren en deze toepassen in verschillende contexten zoals computer graphics en neurale netwerken.
2. object-georiënteerde programmeertechnieken toepassen in de programmeertaal C++.
3. shader-programma's schrijven en uitvoeren op een GPU in GLSL.
4. relevante design patterns onderbouwd selecteren en toepassen.

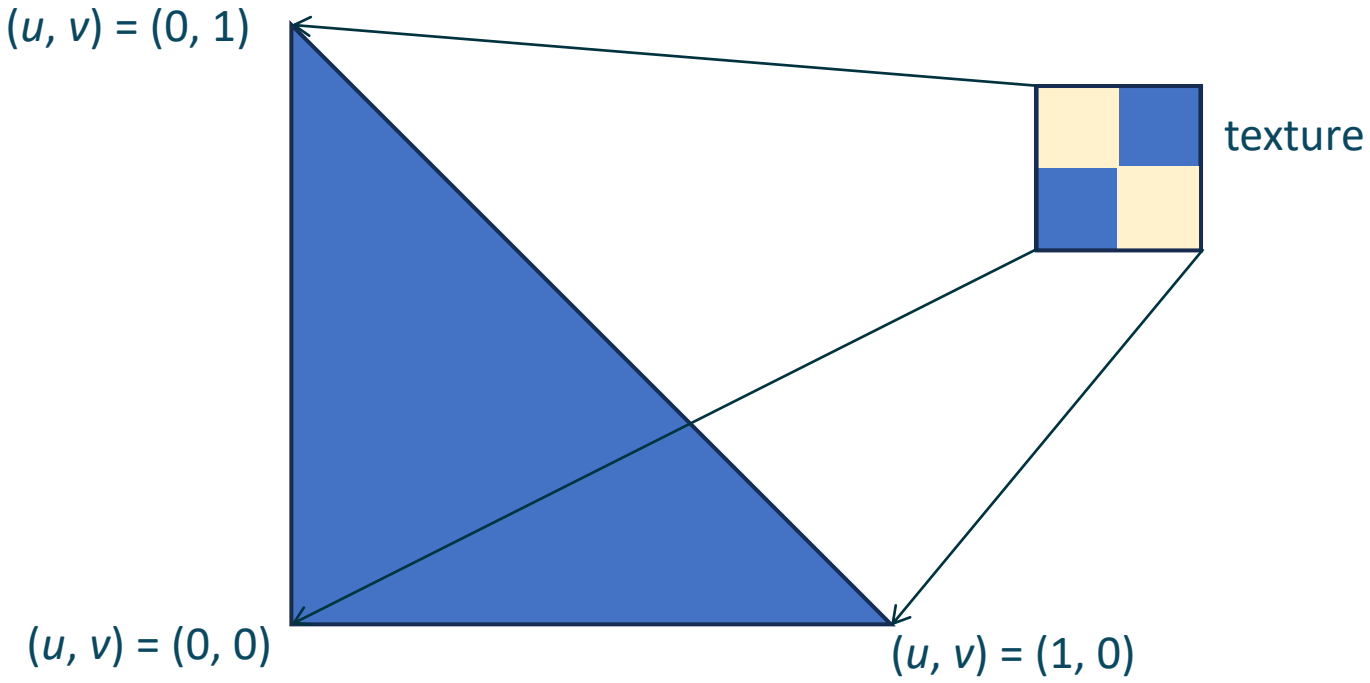
Planning

1. Bespreken opdracht vorige les
2. Textuur
3. Belichting



Texture toevoegen

- Meestal willen we meer details afbeelden op het oppervlak ⇒ *texture mapping*.
- De texture staat in een array van RGB-waarden (bijv. een foto).
- **Probleem**: waar op het oppervlak moet de texture komen?
- **Oplossing**: Voeg texture-coördinaten *u*, *v* toe aan iedere vertex.



Texture toevoegen

- Met texture-coördinaten groter dan 1 wordt de texture als een tegelpatroon herhaald.
- Vertices:

```
GLfloat vertices[] = {
     1,  1,  1, 5, 0, // front top right
     1, -1,  1, 5, 5, // front bottom right
    -1, -1,  1, 0, 5, // front bottom left
    -1,  1,  1, 0, 0, // front top left
     1,  1, -1, 0, 0, // back top right
     1, -1, -1, 0, 5, // back bottom right
    -1, -1, -1, 5, 5, // back bottom left
    -1,  1, -1, 5, 0, // back top left
};
```



Texture toevoegen

- **Vertex shader** krijgt texture-coördinaten binnen als *attribute* en geeft deze door aan de fragment shader met met een `out` variable:

```
#version 330 core
in vec3 pos;
in vec3 col;
in vec2 tex;
out vec3 color = col;
out vec2 texCoord = tex;
      ⋮
```



Texture toevoegen

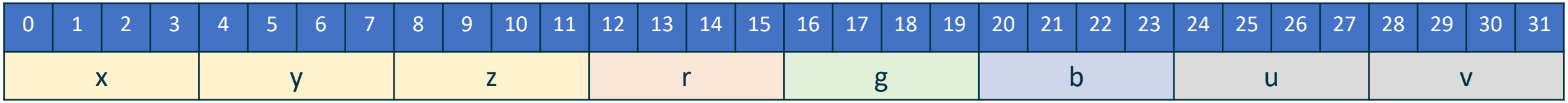
- **Fragment shader** krijgt texture-coördinaten binnen als *attribute* met dezelfde naam als de `out` variable in de vertex shader.
- De texture zelf is een *uniform* van type `sampler2D`.
- Met de functie `texture()` wordt de juiste plek opgezocht:

```
#version 330 core
in vec3 color;
in vec2 texCoord;
out vec4 fragColor;
uniform sampler2D textureSampler;
void main() {
    fragColor = texture(textureSampler, texCoord) * vec4(color, 1.0);
}
```



Texture toevoegen

- **Attributes** `pos`, `col` en `tex` worden gevuld vanuit de vertex buffer.
- 1 vertex:




```
GLint texLocation = glGetAttribLocation(program, "tex");
glVertexAttribPointer(texLocation, 2, GL_FLOAT, GL_FALSE, 32, (void *)24);
glEnableVertexAttribArray(texLocation);
```

- 2 getallen voor texture-coördinaten
- Totale *stride*: 32 bytes (8 32-bit floats)
- *Offset* voor texture-coördinaten: 24 bytes (6 32-bit floats)



Texture toevoegen

- Maak een *texture buffer* om de texture in op te slaan:

```
GLuint texture;
glGenTextures(1, &texture);
glBindTexture(GL_TEXTURE_2D, texture);
```

- Koppel de texture-array aan de texture buffer:

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB,
GL_UNSIGNED_BYTE, pixels);
```

  - Breedte & hoogte
  - Texture-array



- Geef aan hoe de texture geschaald moet worden:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
```



Opdracht 1: Texture

- **Zie instructies op GitLab**.

- Bestudeer het gegeven voorbeeldprogramma.
- Pas de texture-coördinaten van de vertices aan zodat het plaatje maar 1 × op de zijden van de kubus geplakt wordt.
- Vervang de gegenereerde texture door een plaatje uit een bestand.

- Stel, je hebt meerdere kubussen met dezelfde texture. Laad je dan voor iedere kubus de texture opnieuw in het geheugen? Dit neemt een hoop geheugenruimte in beslag.



Pauze (15min)

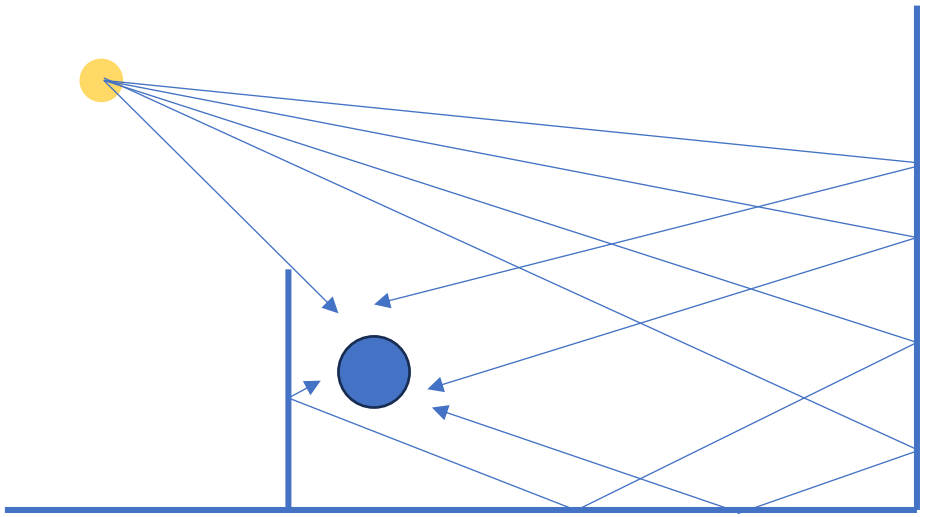
Belichting

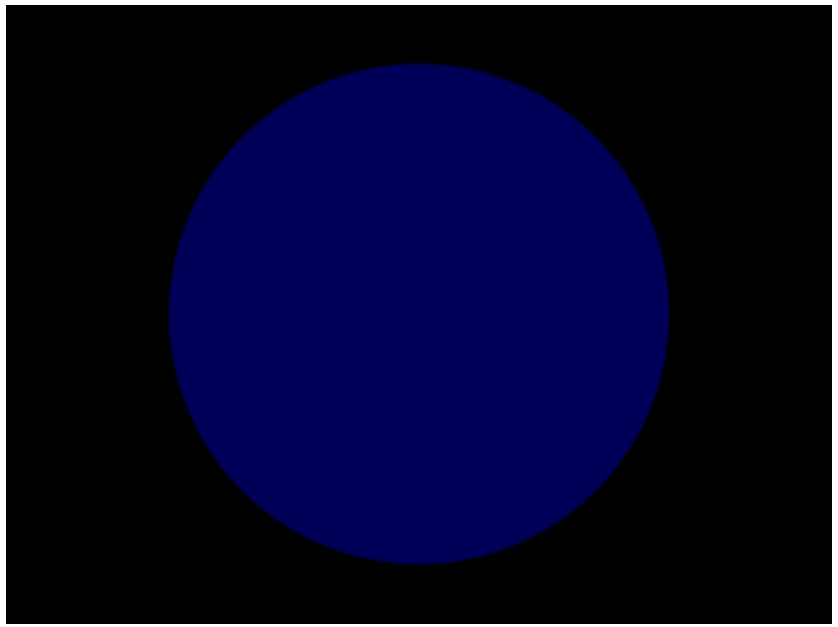
- We zien alleen objecten die licht van een lichtbron reflecteren in ons oog.
- Daarom zijn lichtbronnen essentieel voor een realistische 3D-voorstelling.
- Verschillende soorten belichting:
  - Ambient lighting
  - Diffuse lighting
    - Directional
    - Positional
  - Specular lighting



Ambient lighting

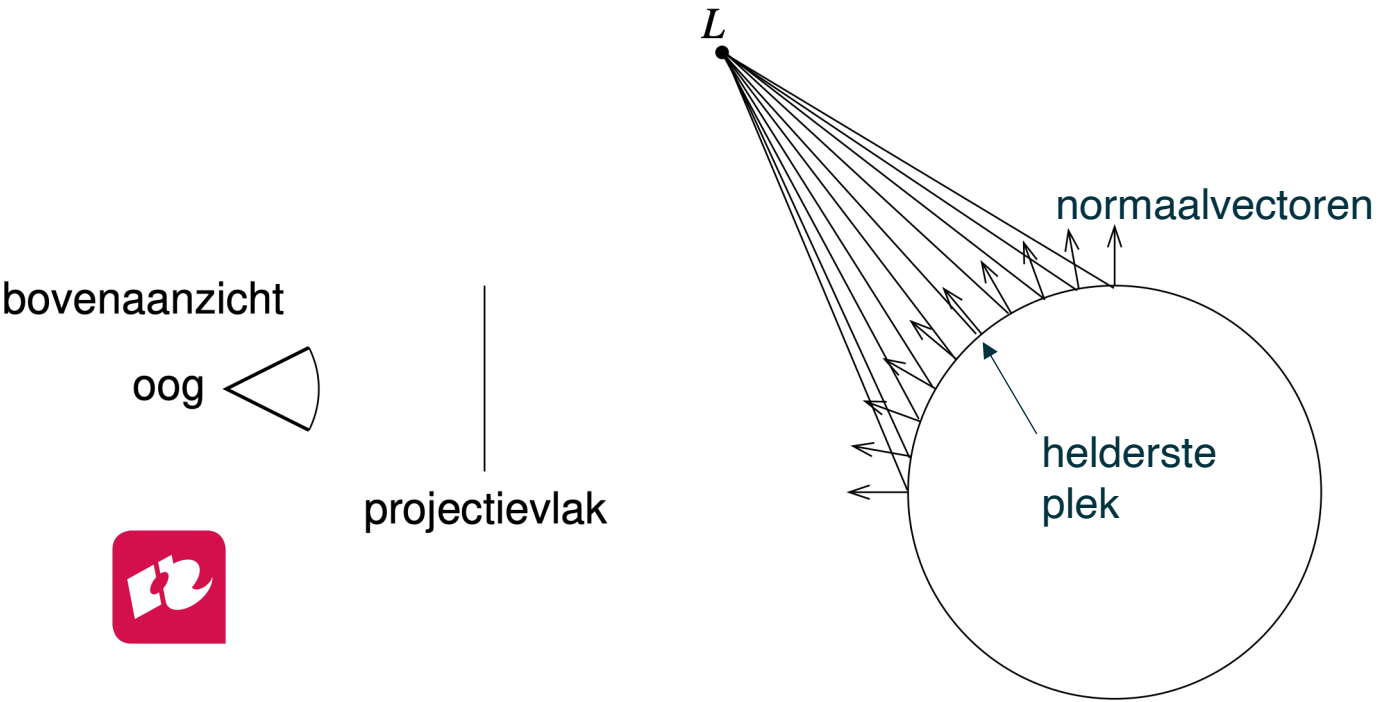
- Licht dat overal “aanwezig” (ambient) is en van alle kanten komt.
- In realiteit: resultaat van heel veel reflecties van allerlei oppervlakken.
- Te complex om precies te modelleren ⇒ simpele benadering.

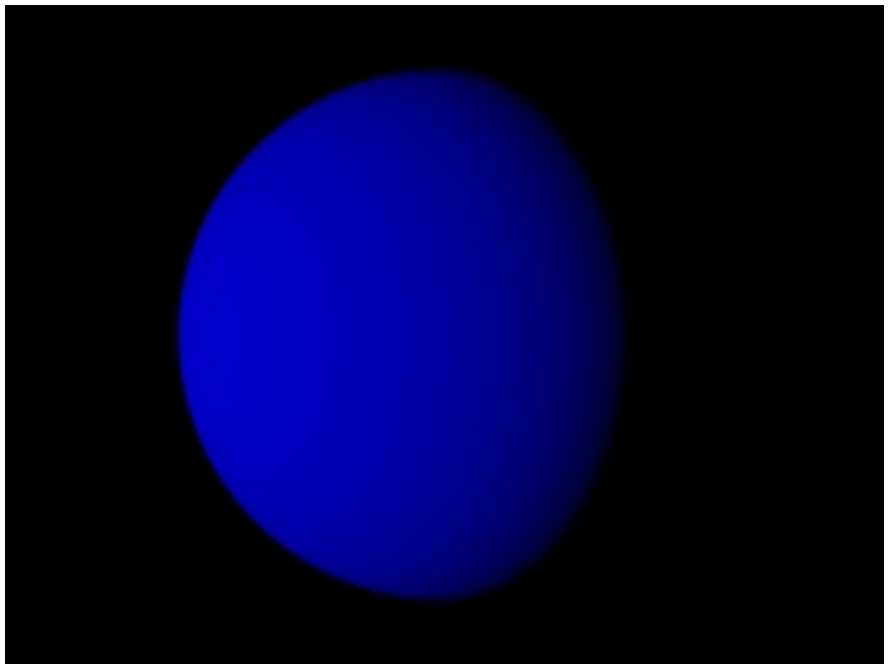




Diffuse lighting

- **Lambertiaans belichtingsmodel**
- Hoe meer een oppervlak naar de lichtbron is toegekeerd, hoe sterker het verlicht wordt.
- Hoe beter de *normaalvector* overeenkomt met de *omgekeerde lichtrichtingsvector*, hoe sterker verlicht.

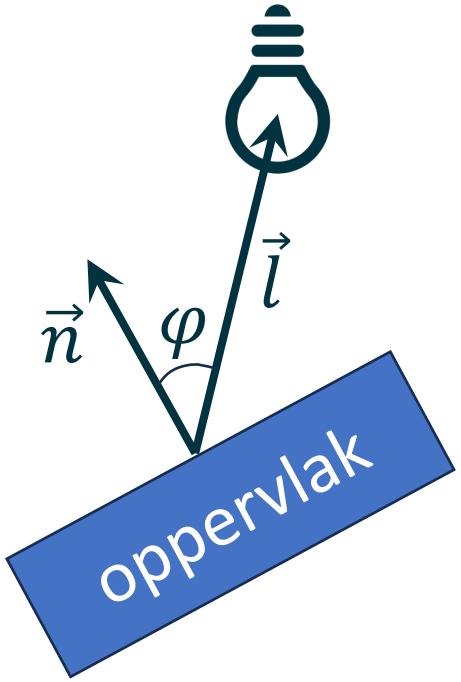




Diffuse lighting

- **Lichtintensiteit I is het hoogste** als de normaalvector van het oppervlak $\vec{n}$ en de omgekeerde lichtvector $\vec{l}$ in dezelfde richting staan.
- We kunnen hiervoor het **inproduct** gebruiken van de twee vectoren:

$$I \propto \vec{n} \cdot \vec{l} = |\vec{n}| \, \left|\vec{l}\right| \cos \varphi$$



- In GLSL is hiervoor de functie `dot()`.
- Om de berekening te laten kloppen moeten we met **eenheidsvectoren** werken (lengte 1).
- In GLSL is hiervoor de functie `normalize()`.

Directional lighting

- Voor *directional lighting* gebruiken we een **vaste lichtvector** (de lichtstralen komen evenwijdig binnen).
- Fragment shader:

```
#version 330 core
in vec3 normal;
vec3 reverseLightDirection = normalize(vec3(2.0f, 1.2f, 1.0f));
vec4 baseColor = vec4(1.0f, 1.0f, 1.0f, 1.0f);
out vec4 fragColor;
void main() {
    vec3 normalizedNormal = normalize(normal);
    float lightness = dot(normalizedNormal, reverseLightDirection);
    fragColor = baseColor;
    fragColor.rgb *= lightness;
}
```



Directional lighting

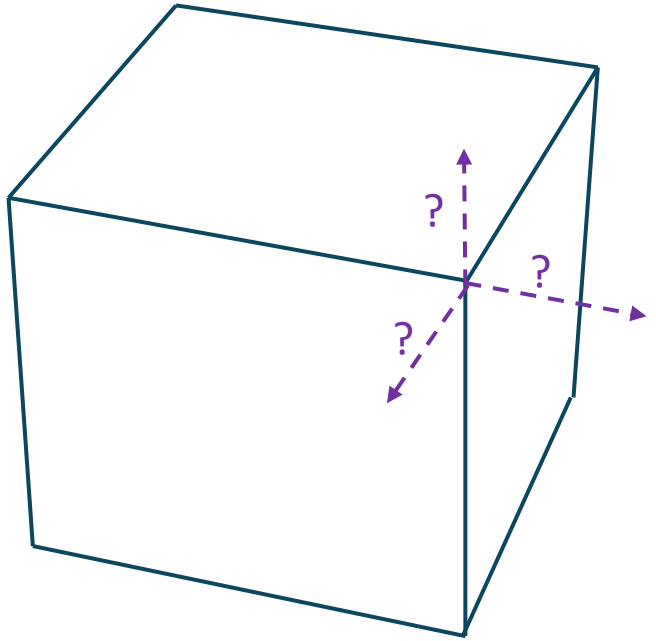
- We hebben dus de normaalvector nodig van ieder fragment.
- Deze wordt doorgegeven door de **vertex shader**.
- Zonder translatie en projectie, alleen rotatie:

```
#version 330 core
in vec3 pos;
in vec3 nor;
out vec3 normal;
⋮
void main() {
    ⋮
    gl_Position = projection * translation * rotation * rotationY * vec4(pos, 1.0);
    normal = mat3(rotation * rotationY) * nor;
}
```



Directional lighting

- We moeten dus bij iedere vertex een normaalvector toevoegen.
- **Probleem**: wat is de normaalvector van een hoekpunt?



- **Oplossing**: voor ieder vlak van de kubus aparte vertices.

Directional lighting

- Totaal 6 vlakken × 2 driehoeken × 3 hoekpunten = 36 vertices:

```
GLfloat vertices[] = {
  // front side
   1,  1,  1, 0, 0, 1, // front top right
   1, -1,  1, 0, 0, 1, // front bottom right
  -1, -1,  1, 0, 0, 1, // front bottom left
  -1, -1,  1, 0, 0, 1, // front bottom left
  -1,  1,  1, 0, 0, 1, // front top left
   1,  1,  1, 0, 0, 1, // front top right
  // right side
   1,  1,  1, 1, 0, 0, // front top right
   1, -1,  1, 1, 0, 0, // front bottom right
   1, -1, -1, 1, 0, 0, // back bottom right
```

Zelfde hoekpunt, andere normaalvector

enz. enz.



Directional lighting

- Voorzie nor-*attribute* van data:

```
GLint norLocation = glGetAttribLocation(program, "nor");
glVertexAttribPointer(norLocation, 3, GL_FLOAT, GL_FALSE, 24, (void *)12);
glEnableVertexAttribArray(norLocation);
```

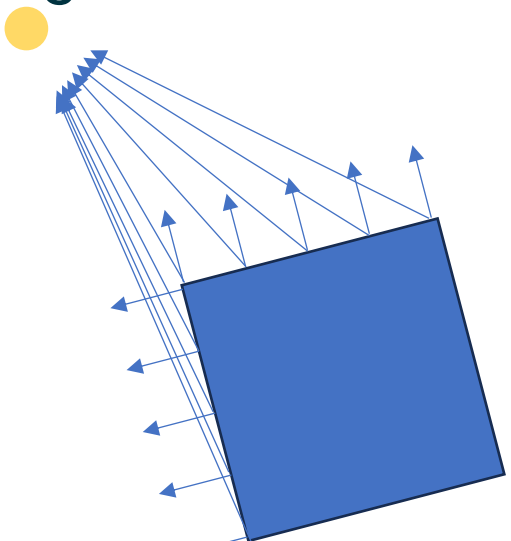
- Vertices worden niet hergebruikt, dus geen ElementArrayBuffer met edges meer nodig.
- Tekeninstructie wordt:

```
glDrawArrays(GL_TRIANGLES, 0, sizeof(vertices) / 6 / sizeof(GLfloat));
```



Positional lighting

- **Probleem**: Directional lighting niet heel realistisch.
  - Alle lichtstralen evenwijdig.
  - Kan alleen als lichtbron oneindig ver weg is.
- **Oplossing**: Positional lighting
  - Lichtbron met een specifieke positie in de 3D-ruimte.
  - `reverseLightDirection` wijst naar die positie.
  - Hangt af van de plek op het oppervlak.
  - Voor ieder fragment nodig: normaalvector, positie in 3D-ruimte.



Positional lighting

- Fragment shader:

```
#version 330 core
in vec3 normal;
in vec3 fragPos;
vec4 baseColor = vec4(1.0f, 1.0f, 1.0f, 1.0f);
vec3 lightPos = vec3(0.2f, 1.2f, 0.0f);
out vec4 fragColor;
void main() {
    vec3 normalizedNormal = normalize(normal);
    vec3 reverseLightDirection = normalize(lightPos - fragPos);
    float lightness = dot(normalizedNormal, reverseLightDirection);
    fragColor = baseColor;
    fragColor.rgb *= lightness;
}
```



Positional lighting

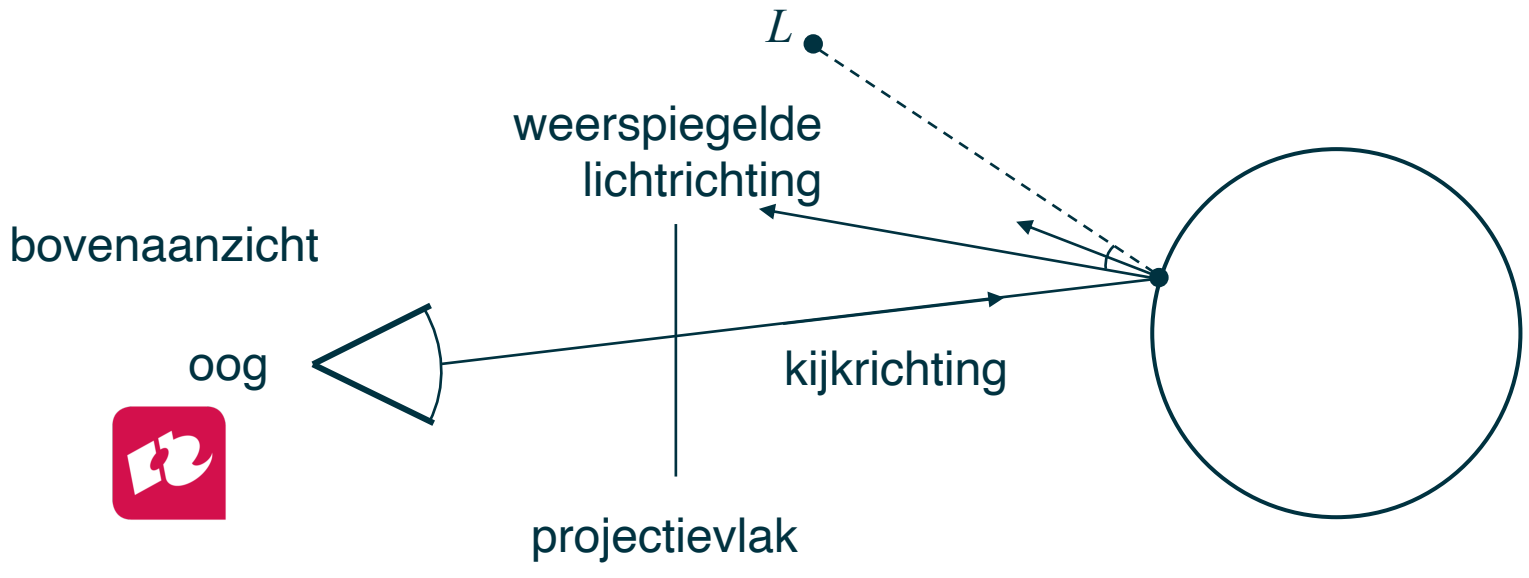
- Vertex shader geeft fragment-positie door aan fragment shader.
- Met rotatie en translatie, zonder projectie:

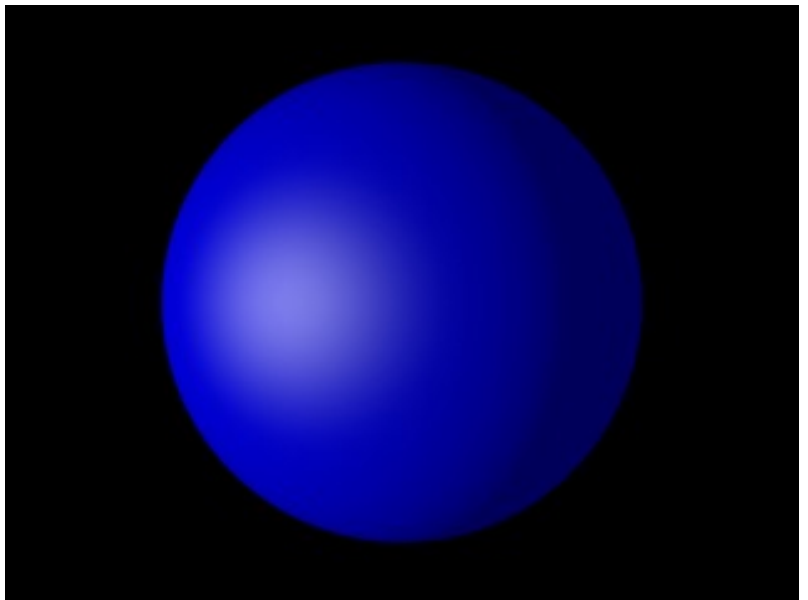
```
#version 330 core
in vec3 pos;
in vec3 nor;
out vec3 normal;
out vec3 fragPos;
⋮
void main() {
    ⋮
    gl_Position = projection * translation * rotation * rotationY * vec4(pos, 1.0);
    normal = mat3(rotation * rotationY) * nor;
    fragPos = vec3(translation * rotation * rotationY * vec4(pos, 1.0));
}
```



Specular lighting

- Glimmende oppervlakken weerspiegelen het licht (*specula* = spiegel).
- Hierdoor onstaat een lichtvlek op het oppervlak.
- De kleur van de vlek is de kleur van de lichtbron.
- Hoe beter de *kijkrichting* overeenkomt met de *omgekeerde weerspiegelde lichtrichting*, hoe sterker verlicht.

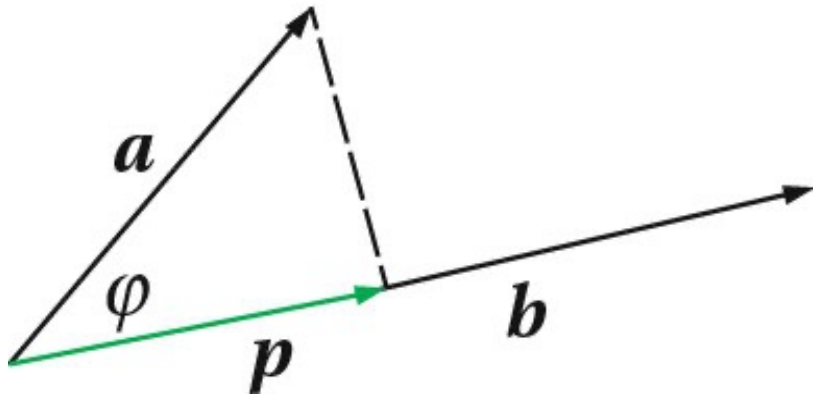




Nodig: projectie van een vector op een andere vector

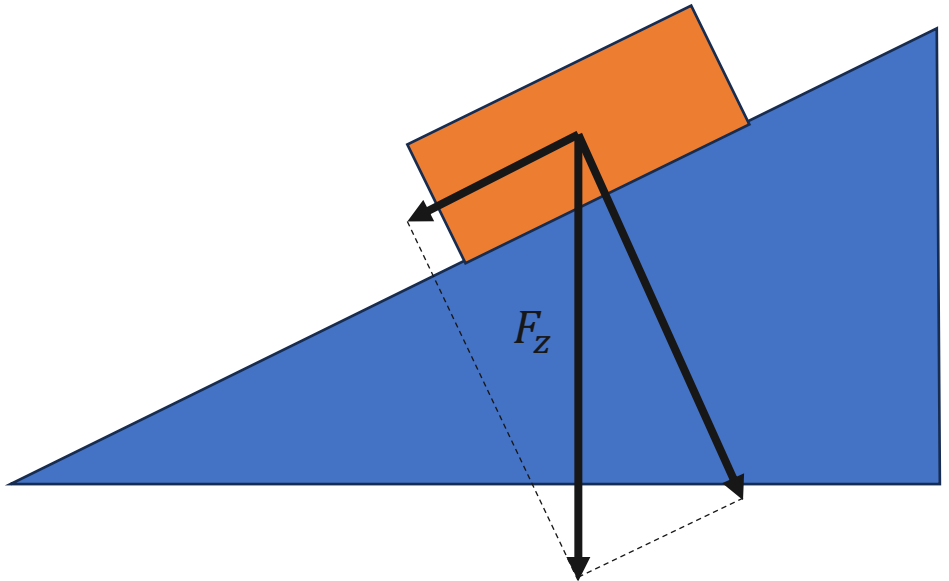
Projectie $\vec{p}$ van vector $\vec{a}$ op vector $\vec{b}$

- $\vec{p}$ heeft dezelfde richting als $\vec{b}$
- Lengte $\vec{p}$: $|\vec{p}| = |\vec{a}| \cos\varphi = |\vec{a}| \frac{\vec{a}\cdot\vec{b}}{|\vec{a}||\vec{b}|} = \frac{\vec{a}\cdot\vec{b}}{|\vec{b}|}$
- $\vec{p} = \frac{|\vec{p}|}{|\vec{b}|}\vec{b} = \frac{\vec{a}\cdot\vec{b}}{|\vec{b}|^2}\vec{b} = \frac{\vec{a}\cdot\vec{b}}{\vec{b}\cdot\vec{b}}\vec{b}$



Andere toepassingen:

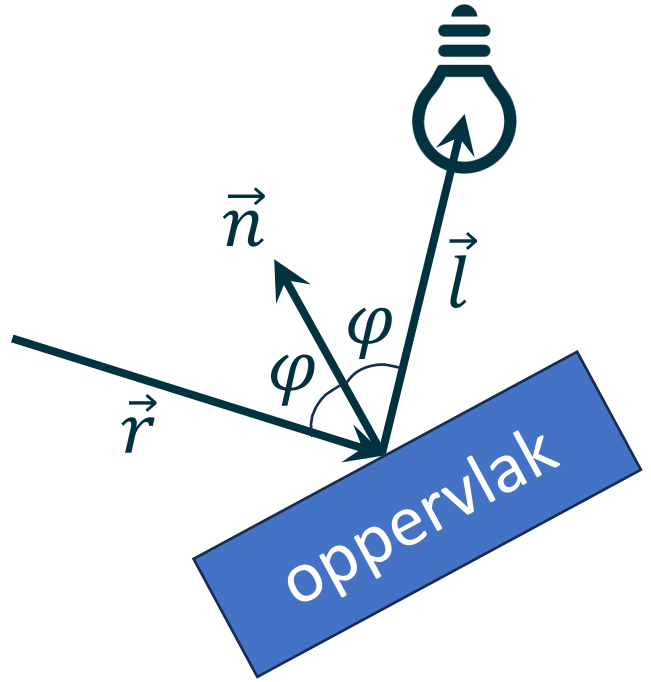
- ontbinden van krachten
- “Schaduw”



Specular lighting

- We hebben dus de *omgekeerde weerspiegelde lichtvector* $\vec{r}$ nodig.

$$\vec{r} = \vec{l} - 2(\vec{n} \cdot \vec{l})\vec{n}$$



- In GLSL is hiervoor de functie `reflect()`.
- We kunnen nu de omgekeerde weerspiegelde lichtvector vergelijken met de *kijkrichting*.
- Dus: inproduct van $\vec{r}$ met kijkrichting.
- Kijkrichting: de eenheidsvector in de richting van de oorsprong naar de positie op het oppervlak: `normalize(fragPos)`.

Specular lighting

- **Fragment shader**:

```
⋮
void main() {
    vec3 normalizedNormal = normalize(normal);
    vec3 reverseLightDirection = normalize(lightPos - fragPos);
    vec3 reflectedDirection = reflect(reverseLightDirection,
      normalizedNormal);
    vec3 viewingDirection = normalize(fragPos);
    float lightness = dot(viewingDirection, reflectedDirection);
    fragColor = baseColor;
    fragColor.rgb *= lightness;
}
```

oog is in de oorsprong

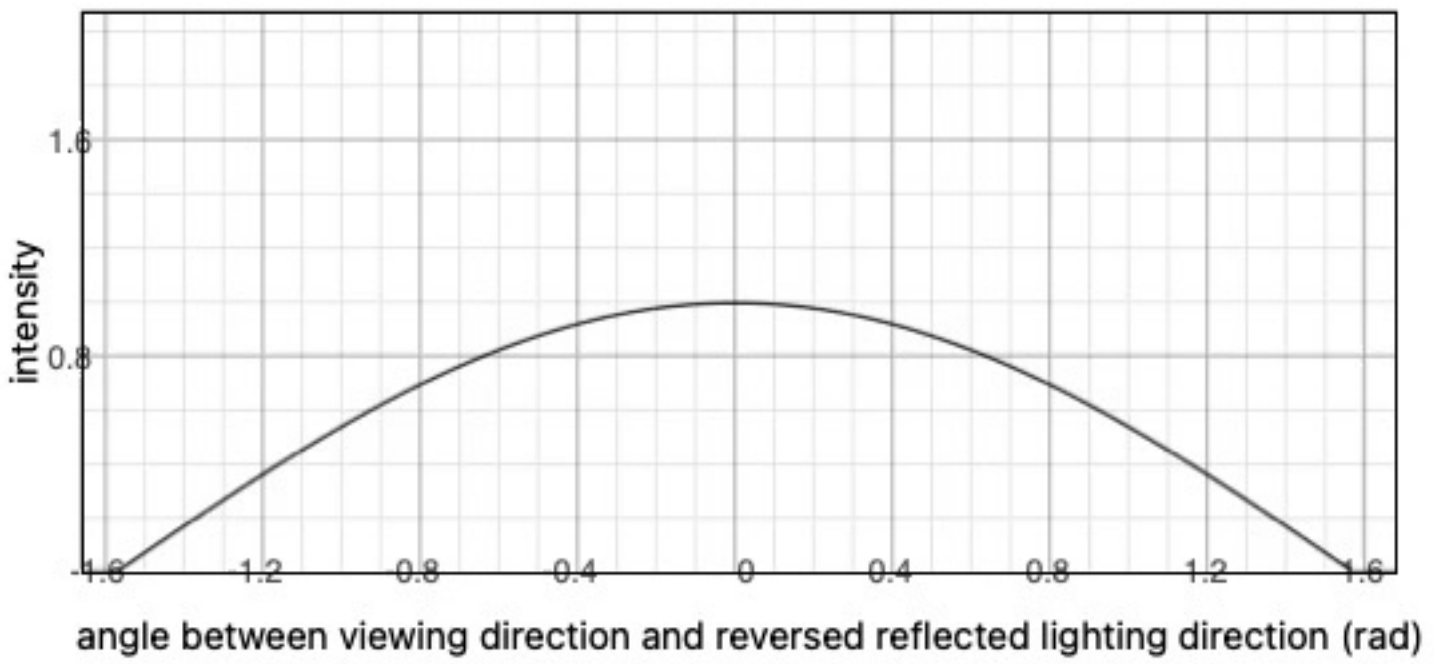


Specular lighting

- Glimvlek is erg groot; niet heel realistisch.
- Meer concentreren: gebruik machtsfunctie:

$$f(x) = x^a$$

- a: *shinyness* (1…32):



Specular lighting: shinyness

- In GLSL: `pow(x, a)`.
- Let op: alleen positieve getallen; gebruik `max(0, x)`.

```
⋮
void main() {
    vec3 normalizedNormal = normalize(normal);
    vec3 reverseLightDirection = normalize(lightPos - fragPos);
    vec3 reflectedDirection = reflect(reverseLightDirection,
      normalizedNormal);
    vec3 viewingDirection = normalize(fragPos);
    float lightness = pow(max(0.0f, dot(viewingDirection,
      reflectedDirection)), 32.0f);
    fragColor = baseColor;
    fragColor.rgb *= lightness;
}
```



Opdracht 2: belichting

- **Zie instructies op GitLab**.

- Combineer de 3 types belichting met een kubus met kleur en texture.



Check-out

- Wat hebben we geleerd?
  - Hoe je textuur toevoegt aan een vorm in een shader.
  - Welke soorten belichting er zijn en hoe die wiskundig werken.
  - Hoe je belichting toevoegt aan een shader.



Afsluiting

Verwerkingsopdracht:

- Leidt de reflectieformule $\vec{r} = \vec{l} - 2(\vec{n} \cdot \vec{l})\vec{n}$ af. Gegeven de normaalvector $\vec{n}$ en de omgekeerde lichtrichting $\vec{l}$, schrijf stap voor stap op hoe je de gereflecteerde vector $\vec{r}$ bepaalt. **Hint**: projecteer $\vec{l}$ op $\vec{n}$ (zie les 2). Dit is afstand van het oppervlak naar de lichtbron, haaks op het oppervlak. Als je nu tweemaal de omgekeerde verplaatsing neemt vanaf de lichtbron, kom je uit op het gevraagde gereflecteerde punt (maak een tekening).

Voorbereidingsopdracht:

- Bereid een korte uitleg voor (max. 5') over elk van de volgende onderwerpen:
  - Texture mapping in OpenGL en de betekenis van *u*, *v*-coördinaten
  - Diffuse belichting (directional en positional)
  - Speculaire belichting, inclusief shinyness

Volgende keer:

- Materiaaleigenschappen

HOGESCHOOL ROTTERDAM

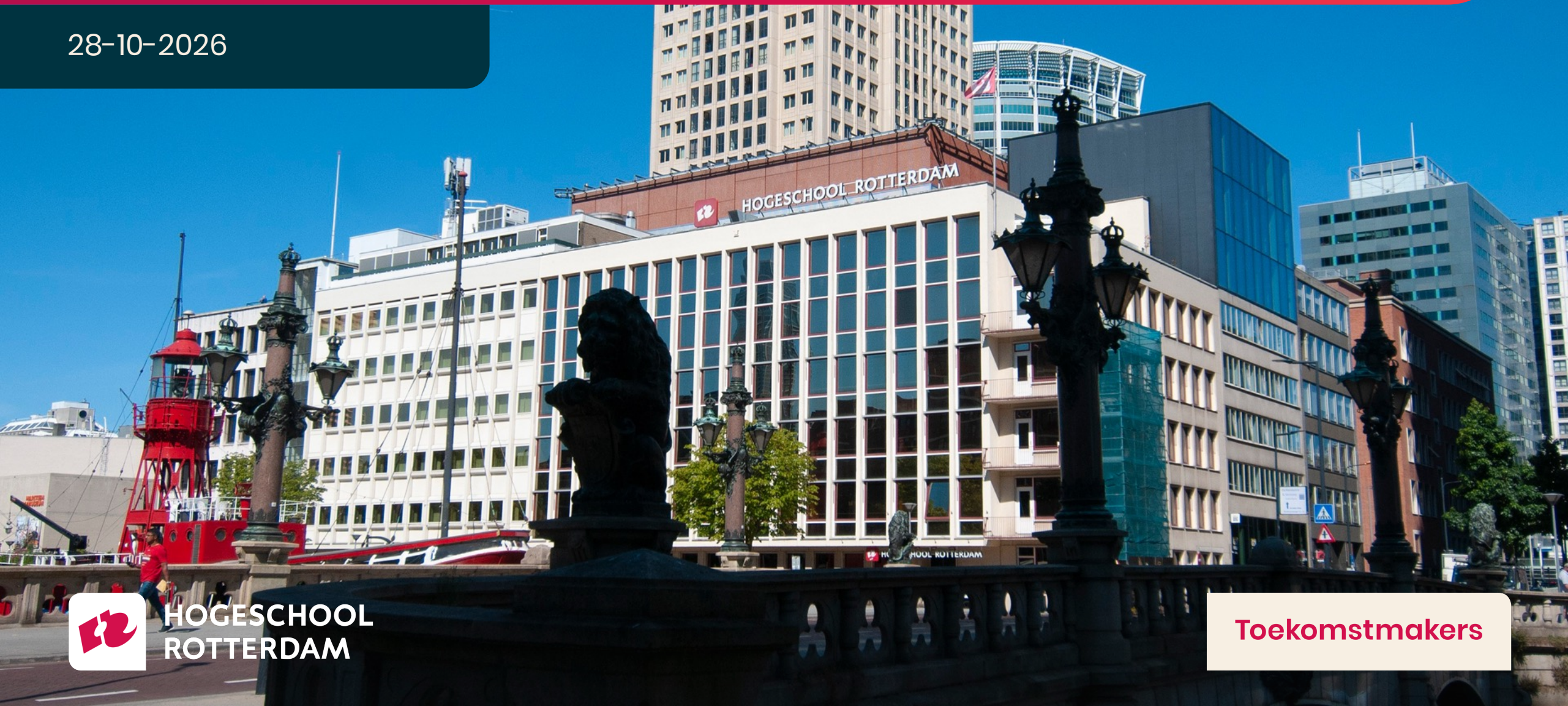
Toekomstmakers

© 2025 Hogeschool Rotterdam, alle rechten voorbehouden

Technische Informatica: High-performance Computing

Les 20: Materiaaleigenschappen

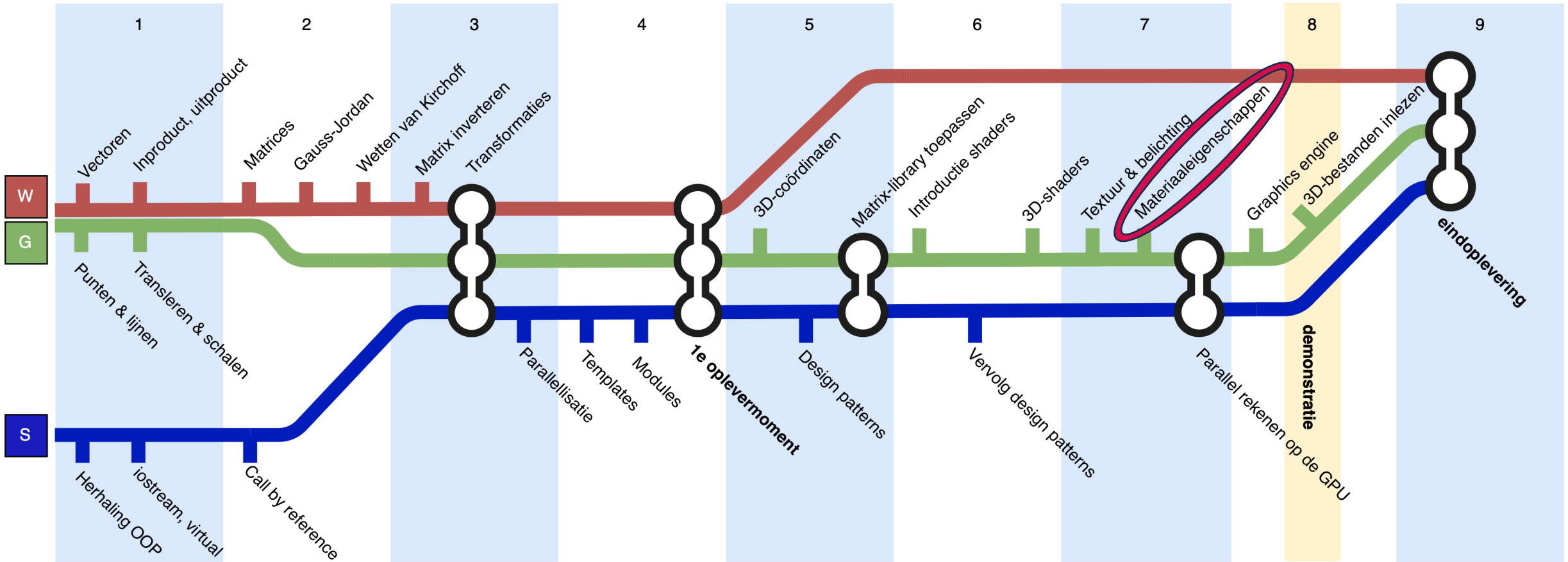
28-10-2026



HOGESCHOOL ROTTERDAM

Toekomstmakers

Metrokaart


1
2
3
4
5
6
7
8
9
W
G
S
Vectoren
Inproduct, uitproduct
Matrices
Gauss-Jordan
Wetten van Kirchoff
Matrix inverteren
Transformaties
Punten & lijnen
Transleren & schalen
Herhaling OOP
iostream, virtual
Call by reference
Parallellisatie
Templates
Modules
1e oplevermoment
3D-coördinaten
Design patterns
Matrix-library toepassen
Introductie shaders
Vervolg design patterns
3D-shaders
Textuur & belichting
Materiaaleigenschappen
Parallel rekenen op de GPU
demonstratie
Graphics engine
3D-bestanden inlezen
eindoplevering

Leerdoelen

De student kan:

1. bewerkingen met vectoren en matrices uitvoeren en deze toepassen in verschillende contexten zoals computer graphics en neurale netwerken.
2. object-georiënteerde programmeertechnieken toepassen in de programmeertaal C++.
3. shader-programma's schrijven en uitvoeren op een GPU in GLSL.
4. relevante design patterns onderbouwd selecteren en toepassen.

Planning

1. Materiaaleigenschappen
2. Transparency, reflectivity, diffusivity
3. Structuur suggereren met de normaalvector



Materiaaleigenschappen

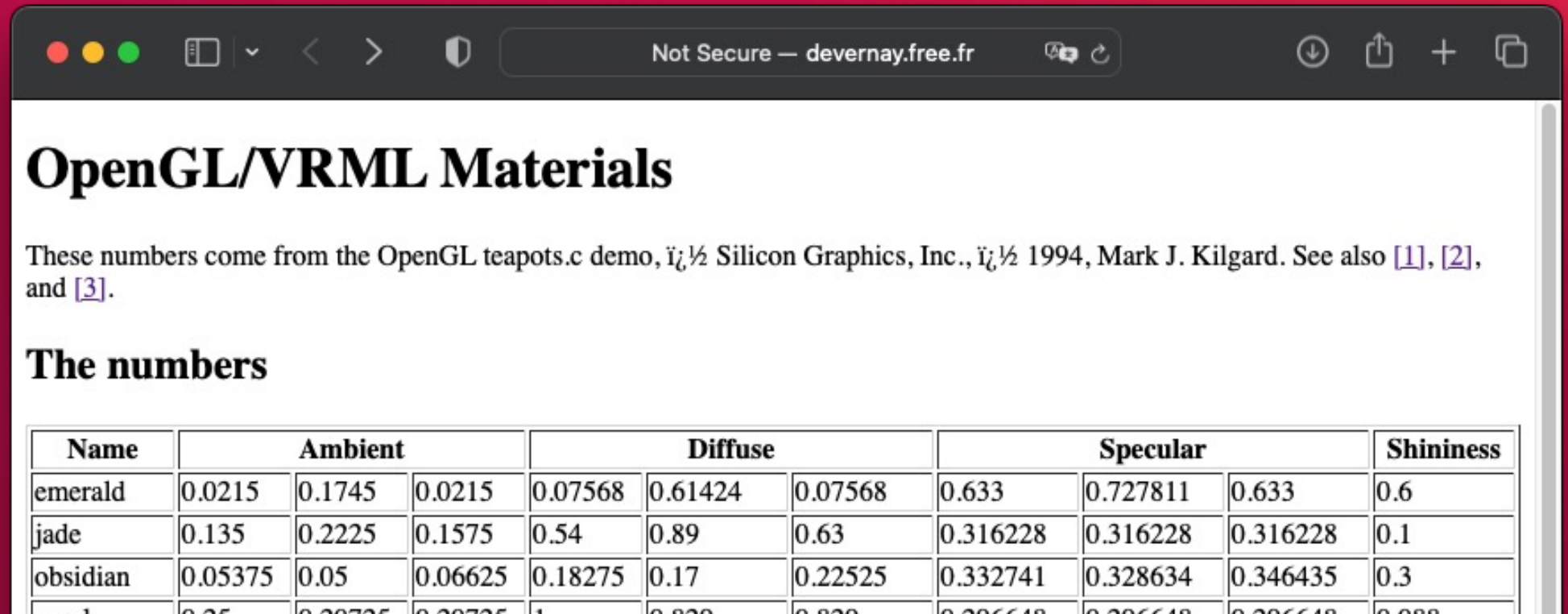
- **Vorige les**: 3 soorten belichting: ambient, diffuse, specular
- **Nu**: materiaal nabootsen met voor iedere soort een andere RGB-kleur.
  - Gecombineerd met de RGB-kleur van de lichtbron.
  - Shinyness
- **Straks**: Transparency
- **Optioneel**: Bi-directional Reflectance Distribution Function (BRDF)



Opdracht 1: materiaaleigenschappen

- **Zie instructies op GitLab**.

- Voeg materiaaleigenschappen toe aan je shader.
- Experimenteer met verschillende materialen.

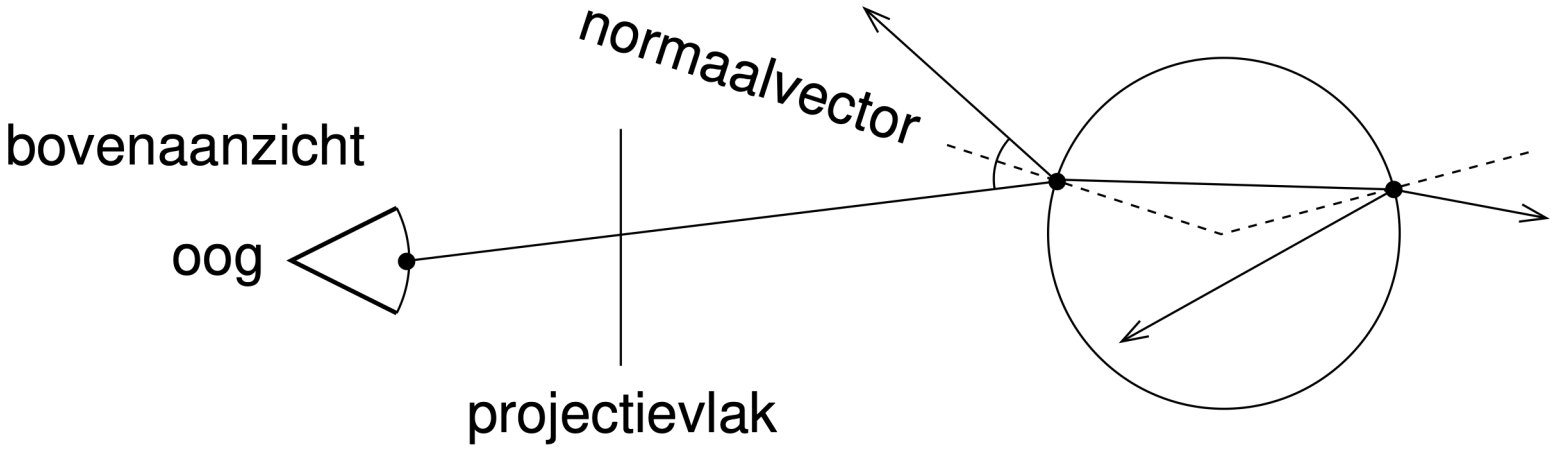


Pauze (15min)

Transparency, reflectivity, diffusivity

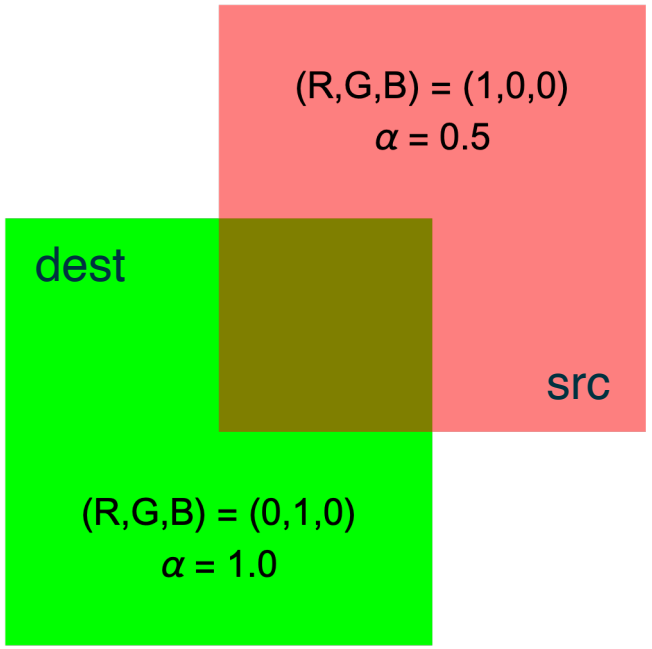
- Sommige materialen zijn **doorschijnend** of **spiegelend** of allebei.
- Als licht van het ene medium naar het andere gaat, **breekt** het.
- Licht kan **diffuus verstrooid** worden door een medium.
- Dit maakt de berekeningen al gauw zeer complex.
- Wij beperken ons hier tot doorschijnendheid (*transparency*).





Transparency

- α-waarde beschrijft opaciteit (ondoorzichtigheid).
  - α = 0: transparant
  - α = 1: opaak (ondoorzichtig)
- De kleur van een pixel is een combinatie van de te tekenen kleur (source) en de kleur die al in de buffer zit (destination).



$$\begin{pmatrix} R \\ G \\ B \\ \alpha \end{pmatrix} = \alpha_s \begin{pmatrix} R_s \\ G_s \\ B_s \\ \alpha_s \end{pmatrix} + (1 - \alpha_s) \begin{pmatrix} R_d \\ G_d \\ B_d \\ \alpha_d \end{pmatrix}$$

$$0.5 \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0.5 \end{pmatrix} + (1 - 0.5) \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1.0 \end{pmatrix} = \begin{pmatrix} 0.5 \\ 0.5 \\ 0 \\ 0.75 \end{pmatrix}$$

Opdracht 2: Transparency

- **Zie instructies op GitLab**.

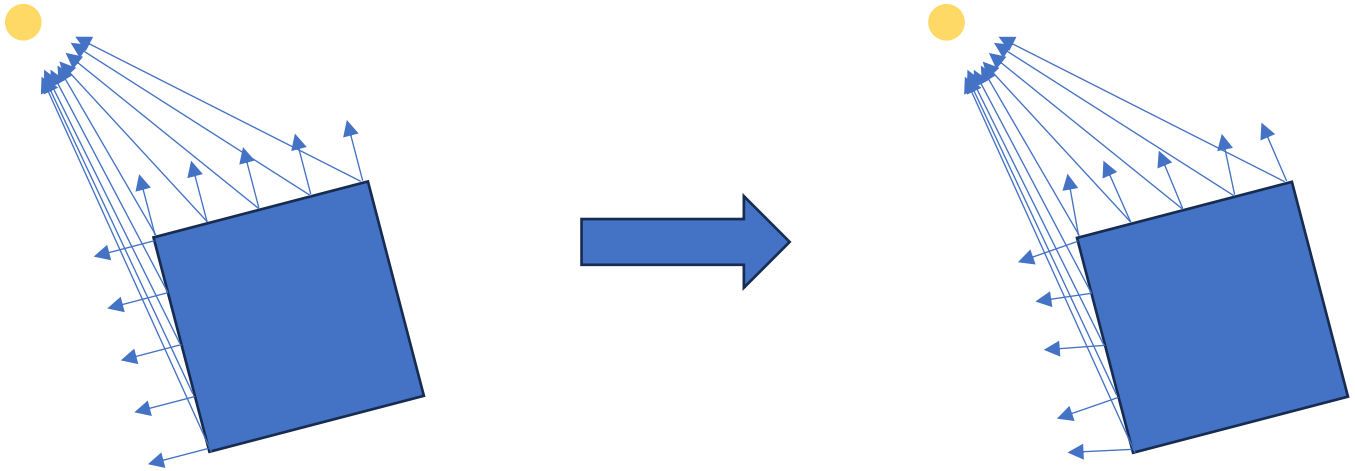
- Voeg transparency toe aan je shader.
- Stel hiervoor de source- en destination-factoren van de *blend function* in.
- Experimenteer met verschillende achtergrondkleuren.



Pauze (15min)

Structuur suggereren met de normaalvector

- Normaalvector wordt gebruikt om belichting te berekenen.
- Wat als we die gaan manipuleren?
- Oppervlak blijft vlak, maar *lijkt* gebobbeld.



- Bijv.:

```
vec3 disturbedNormal = normal + 0.1 * sin(10.0 * pos);
vec3 normalizedNormal = normalize(disturbedNormal);
```



Opdracht 3: Normaalvector manipuleren

- **Zie instructies op GitLab**.

- Bekijk het effect van kleine verstoringen in de normaalvector.
- Voeg hiervoor een verstoring aan de normaalvector toe op basis van de ongetransformeerde 3D-positie.



Check-out

- Wat hebben we geleerd?
  - Hoe materiaaleigenschappen wiskundig nagebootst kunnen worden.
  - Hoe transparantie werkt.
  - Hoe je oppervlaktestructuur kunt suggereren door de normaalvector te manipuleren.

Afsluiting

Verwerkingsopdracht:

- Zorg dat je de 3 opdrachten van deze les helemaal af hebt en getest.

Voorbereidingsopdracht:

- A
- B

Volgende keer:

- Parallel computing op de GPU



HOGESCHOOL ROTTERDAM

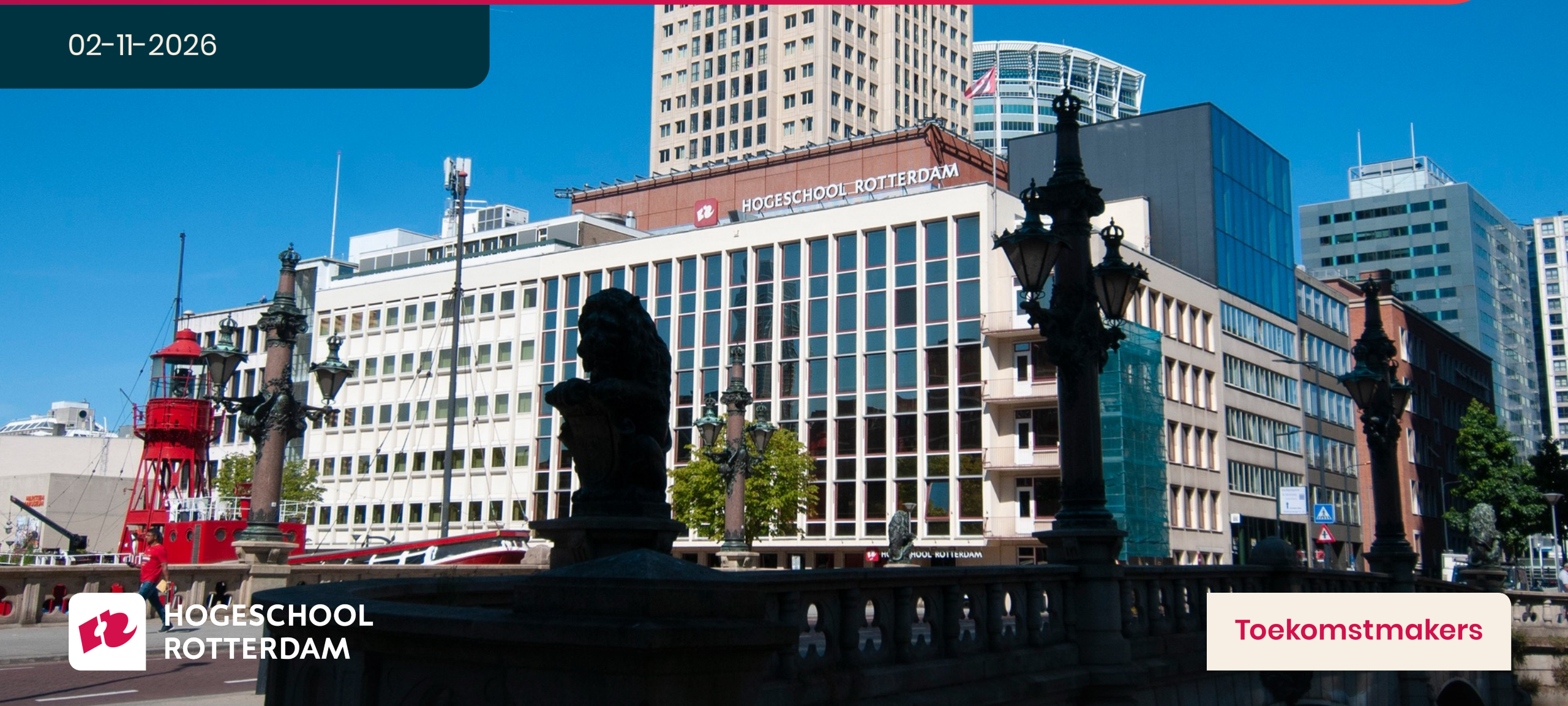
Toekomstmakers

© 2025 Hogeschool Rotterdam, alle rechten voorbehouden

Technische Informatica: High-performance Computing

Les 22: Graphics engine: alles samenvoegen

02-11-2026

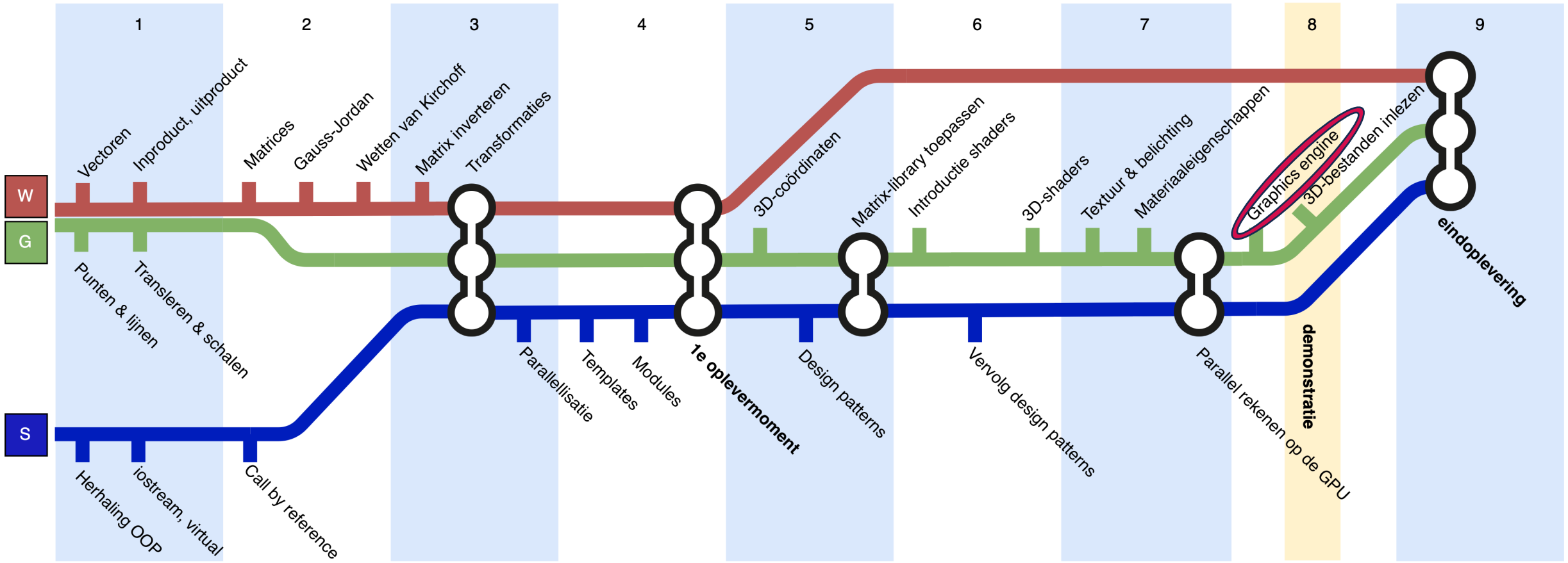


HOGESCHOOL ROTTERDAM

Toekomstmakers

Quiz

Metrokaart


1
2
3
4
5
6
7
8
9
W
G
S
Vectoren
Inproduct, uitproduct
Matrices
Gauss-Jordan
Wetten van Kirchoff
Matrix inverteren
Transformaties
Punten & lijnen
Transleren & schalen
Herhaling OOP
iostream, virtual
Call by reference
Parallellisatie
Templates
Modules
1e oplevermoment
3D-coördinaten
Design patterns
Matrix-library toepassen
Introductie shaders
Vervolg design patterns
3D-shaders
Textuur & belichting
Materiaaleigenschappen
Parallel rekenen op de GPU
Graphics engine
demonstratie
3D-bestanden inlezen
eindoplevering

Leerdoelen

De student kan:

1. bewerkingen met vectoren en matrices uitvoeren en deze toepassen in verschillende contexten zoals computer graphics en neurale netwerken.
2. object-georiënteerde programmeertechnieken toepassen in de programmeertaal C++.
3. shader-programma's schrijven en uitvoeren op een GPU in GLSL.
4. relevante design patterns onderbouwd selecteren en toepassen.



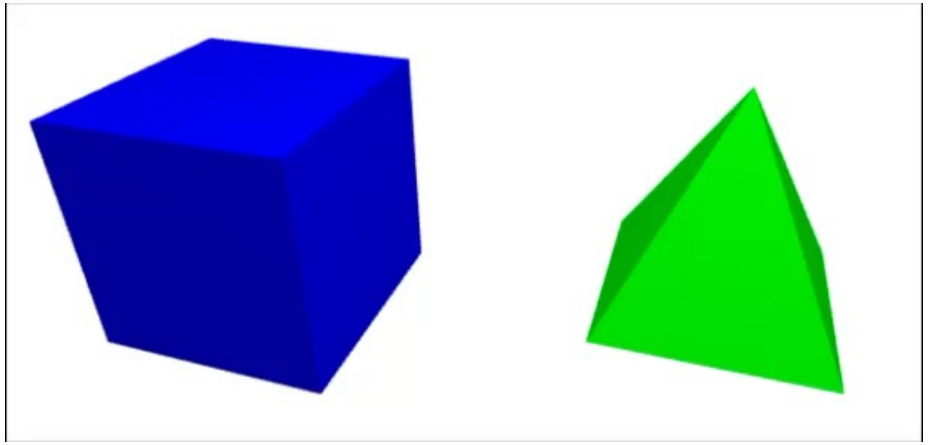
Planning

1. Meerdere vormen
2. Eigenschappen
3. Camera
4. Lichtbron
5. Muis-input



Meerdere vormen

- **Graphics engine**: meerdere vormen kunnen toevoegen
- Iedere vorm:
  - VBO: serie vertices (posities, normaalvectoren, evt. texture-coördinaten)
  - VAO: structuur van de VBO (welk datatype, welke getallen wat betekenen)
  - transformaties (rotaties, schaling, translatie)
  - eigenschappen (kleur, textuur, reflectie, enz.)
- Voor alle vormen: zelfde shader-programma
  - Data voor *uniforms* in de shader kan per vorm verschillen



C++ programma

Eenmalig:

1. Shaders compileren.
2. Voor iedere vorm:
    1. VAO aanmaken en binden
    2. VBO aanmaken en binden.
    3. de VBO vullen met data.
    4. *attributes* uit de vertex shader koppelen aan getallen in de VBO.

Telkens weer, voor iedere vorm:

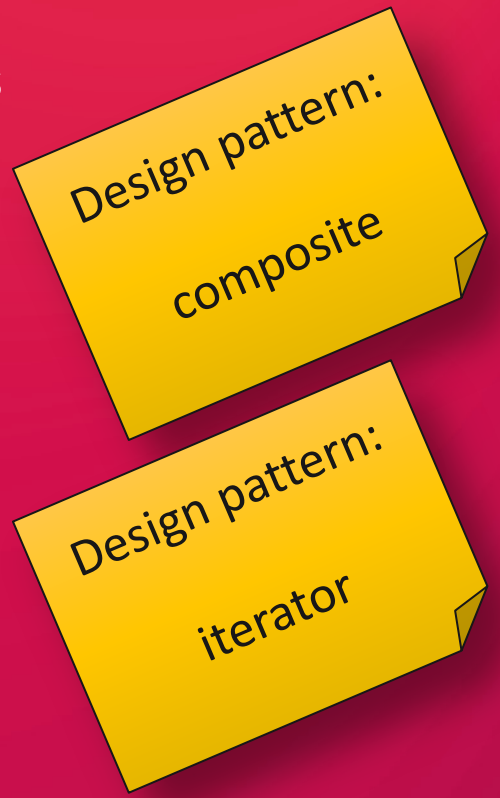
1. De VAO voor deze vorm binden.
2. De *uniforms* van data voorzien voor deze vorm.
3. De vorm tekenen met `glDrawArrays()`.



Opdracht 1: Meerdere vormen (object-georiënteerd)

- **Zie instructies op GitLab**.

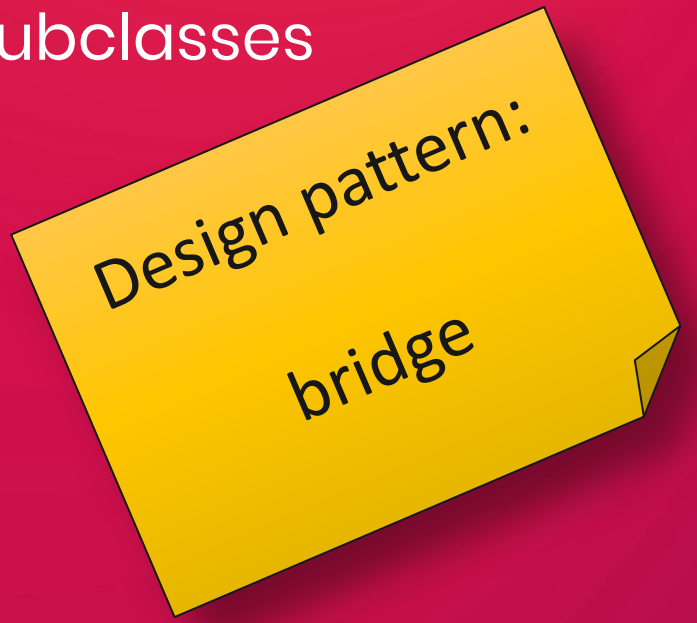
- Voorbeeldcode laat zien hoe je twee vormen rendert (niet-OOP).
- Pas het programma aan zodat het OOP gebruikt.
- Maak een abstracte class `Shape` met concrete subclasses `Cube`, `Pyramid`, `Sphere` en `Cylinder`.
- De scene is *samengesteld* uit meerdere vormen die in een lijst staan. Het renderen van iedere vorm wordt uitbesteed aan de vorm zelf.
- De lijst zou bijv. ook een boomstructuur kunnen zijn. Als er maar een manier is om door alle vormen heen te lopen.



Opdracht 2: Eigenschappen

- **Zie instructies op GitLab**.

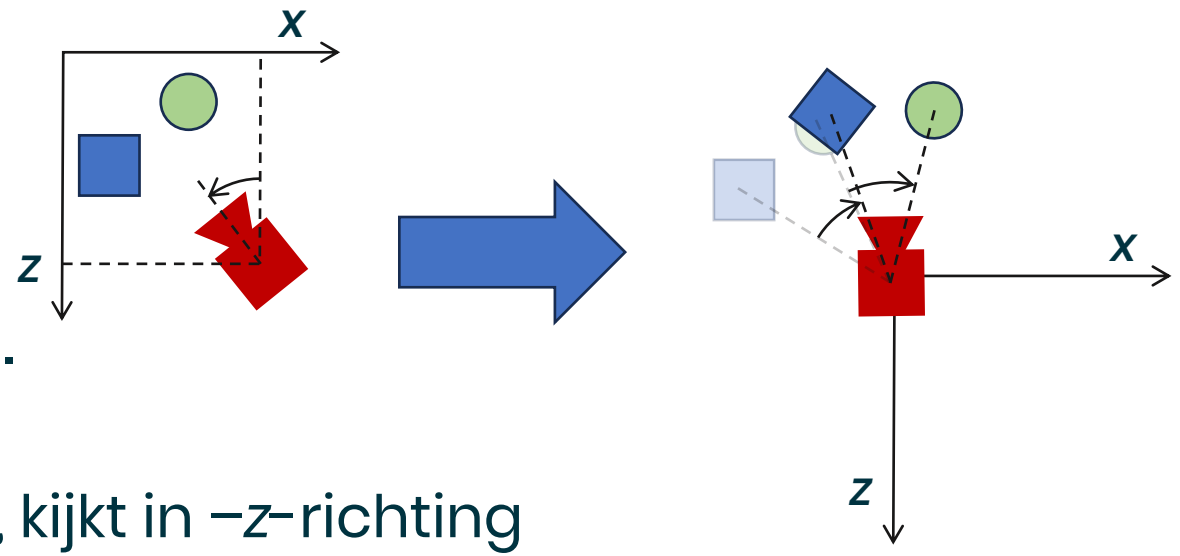
- Aparte classes voor blauwe kubus, blauwe bol, rode kubus, rode bol, enz.?
- Nee, één soort class voor vormen en één soort class voor eigenschappen.
- Maak een abstracte class `Property` met concrete subclasses als `Color`, `Texture`, `DiffuseReflection`, enz.
- Zorg dat je meerdere eigenschappen aan een vorm kan toevoegen.



Camera

- Tot nu toe: vaste projectie.
- Vanaf nu:
  - positie
  - rotatie
  - field of view

  van de camera kunnen instellen.



- Wiskundig gezien:
  - camera altijd in de oorsprong, kijkt in $-z$-richting
  - positie instellen door alle vormen te transleren in omgekeerde richting
  - rotatie instellen door alles rond de camera te laten roteren
- `Camera` is ook een soort transformatie ⇒ subclass van `Transform`

Camera-transformatie

- **Camera-transformatie** bestaat uit:
  1. Rotaties rond de *x*-, *y*-, en *z*-as.
  2. Translatie met een vector die tegengesteld is aan de positievector van de camera.
  3. Perspectiefprojectie met een gegeven *field of view*.
- Voor **posities**: alle 3 de onderdelen toepassen in vertex shader.
- Voor `fragPos` (voor lichtberekeningen): alleen 1 en 2.
- Voor **normaalvectoren**: alleen 1.

- Zorg dus dat de 3 onderdelen apart van elkaar opgeslagen worden in de `Camera`-class.



Opdracht 3: Camera-class

- **Zie instructies op GitLab**.

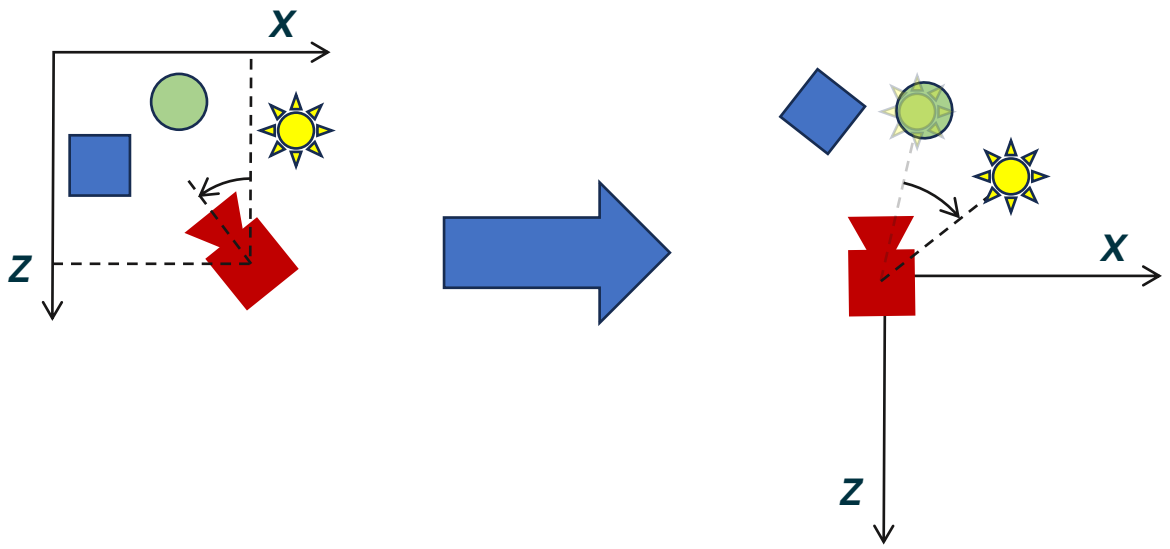
- Voeg een `Camera`-class (subclass van `Transform`) toe aan je programma uit de vorige opdracht.
- Voeg een constructor toe waar je de (x, y, z)-positie, rotatiehoeken rond de x-, y-, en z-as, en een *field of view* aan kan meegeven.
- De verschillende vormen hoeven nu niet meer apart “naar achteren” gezet te worden; dit wordt door de camera-transformatie gedaan.
- Pas de shaders en het C++-programma hierop aan.



Pauze (15min)

Lichtbron

- We willen voor de graphics engine de positie en de kleur van de lichtbron ook kunnen instellen.
- **Belangrijk**: de lichtbron moet ook getransformeerd worden door de camera-transformatie (zonder projectie).



Opdracht 4: Lichtbron-class

- **Zie instructies op GitLab**.

- Voeg een `Lightsource`-class toe aan je programma uit de vorige opdracht.
- Voeg een constructor toe waar je de (x, y, z)-positie en de kleur van de lichtbron kunt meegeven.
- Pas de shaders en het C++-programma hierop aan.

- Extra: kun je meerdere lichtbronnen toevoegen aan je programma?



Muis-input

- **Wens**: de camera (of andere onderdelen) met de muis besturen.
- In GLFW: `glfwGetCursorPos()` geeft *x*- en *y*-coördinaten van de muiscursor (voor rondkijken).
- Voor scroll-wiel (voor voor- en achteruit bewegen) is een callback-functie nodig:

```
double scroll;
void scrollCallBack(GLFWwindow* window, double xoffset, double yoffset) {
  scroll = yoffset;
}
```

- Instellen met:

```
glfwSetScrollCallback(window, scrollCallBack);
```



Opdracht 5: Muis-input

- **Zie instructies op GitLab**.

- Voeg muis-input toe aan je programma uit de vorige opdracht.
- Pas de camera-rotatie en -positie aan op basis van de muisbeweging en scroll-wiel.
- Zie https://www.glfw.org/docs/3.3/input_guide.html#input_mouse voor meer informatie.



Check-out

- Wat hebben we geleerd?
  - Hoe je meerdere vormen rendert met een shader.
  - Hoe je een camera en een lichtbron toevoegt aan je graphics engine.
  - Hoe je de muis kunt gebruiken om de camera te besturen.



Afsluiting

Verwerkingsopdracht:

- Zorg dat je alle 5 de opdrachten afhebt.

Voorbereidingsopdracht:

- Bedenk hoe je alle features van je graphics engine kan demonsteren.
- Installeer Blender op je computer.

Volgende keer:

- 3D-bestanden inlezen



HOGESCHOOL ROTTERDAM

Toekomstmakers

© 2025 Hogeschool Rotterdam, alle rechten voorbehouden